

**COLLEGE OF TECHNOLOGY AND ENGINEERING**

**MAHARANA PRATAP UNIVERSITY OF  
AGRICULTURE & TECHNOLOGY**

**UDAIPUR (RAJ.)**



**A**

**PROJECT REPORT**

**ON**

**Stocks & Commodities Price Forecasting Engine**

submitted in partial fulfillment for the award of the Degree of Bachelor of  
Technology in

**Department of Computer Science & Engineering**

(Session 2025-2026)

**Submitted To:**

**Dr. Kalpana Jain**

Head of Department

Computer Science & Engineering

**Submitted By:**

Harshvardhan Soni

Hussain Attari

Mohit Joshi

Mukul Dangi

Raghav Sanadhya

# ACKNOWLEDGEMENT

It is a well-known fact the talent of a person is to be nurtured properly to obtain the best result for a person, the main requirement is proper guidance along with the right direction so that the desired goal can be achieved. Making optimum use of each opportunity is very important.

First of all, we gave Thank all those people who helped us with their guidance and assistance, without which this project would not be successful.

We had acknowledged the kind grace of our dean for providing us with a great opportunity by giving me full support and cooperation.

I am very grateful to **Dr. Kalpana Jain (H.O.D, C.S.E) (Project In-charge)** College of Technology and Engineering, Udaipur (Raj.) who actively supported our Project by letting us select the project report and offered us guidance.

I cannot express my thanks in words to my parents, friends and my family members for their great support and love.

# **DECLARATION**

We hereby declare that the Project titled "Stocks & Commodities Price Forecasting Engine" has been developed by us and is not reproduced as-is from any other source. It has been submitted in partial fulfillment of the requirement for the award of Bachelor of Technology in Computer Science and Engineering MPUAT Udaipur, and has not been submitted anywhere else for the award of any other degree.

Date: 11/05/2026

Place: Udaipur

## **Name of Students:**

Harshvardhan Soni

Hussain Attari

Mohit Joshi

Mukul Dangi

Raghav Sanadhya

# COLLEGE OF TECHNOLOGY AND ENGINEERING

MAHARANA PRATAP UNIVERSITY OF AGRICULTURE & TECHNOLOGY

UDAIPUR (RAJ.)



## CERTIFICATE

This is to certify that the project entitled "Stocks & Commodities Price Forecasting Engine" has been completed and submitted by Harshvardhan Soni ,Hussain Attari, Mohit Joshi, Mukul Dangi, Raghav Sanadhya in partial fulfillment of the requirement for the award of Bachelor of Technology in Computer Science and Engineering from College of Technology and Engineering, a constituent of Maharana Pratap University of Agriculture and Technology, Udaipur.

**Dr. Kalpana Jain**

Head of Department  
Department of Computer Science and Engineering  
College of Technology and Engineering, Udaipur

# TABLE OF CONTENTS

|                                                           |    |
|-----------------------------------------------------------|----|
| LIST OF FIGURES .....                                     | v  |
| ABBREVIATIONS .....                                       | vi |
| ABSTRACT.....                                             | 1  |
| 1. INTRODUCTION .....                                     | 3  |
| 1.1 Context and Background.....                           | 3  |
| 1.2 Problem Statement .....                               | 4  |
| 1.3 Motivation .....                                      | 4  |
| 1.4 Types of Forecasting Approaches .....                 | 5  |
| 1.4.1 Statistical Methods .....                           | 5  |
| 1.4.2 Machine Learning Methods.....                       | 5  |
| 1.4.3 Deep Learning Methods .....                         | 5  |
| 1.4.4 Ensemble Methods .....                              | 6  |
| 1.5 Scope and Objectives of the Project.....              | 6  |
| 2. LITERATURE SURVEY .....                                | 8  |
| 2.1 Overview .....                                        | 8  |
| 2.2 Gradient Boosting for Financial Forecasting .....     | 8  |
| 2.3 Ensemble Methods for Price Prediction.....            | 9  |
| 2.4 Feature Engineering for Financial Time-Series .....   | 10 |
| 2.5 Backtesting and Model Evaluation .....                | 10 |
| 2.6 Relevant Prior Work and Gaps.....                     | 11 |
| 3. METHODOLOGY .....                                      | 12 |
| 3.1 System Architecture Overview .....                    | 12 |
| 3.2 Asset Universe.....                                   | 12 |
| 3.3 Data Collection and Caching (data_loader.py) .....    | 13 |
| 3.4 Feature Engineering (features.py) .....               | 15 |
| 3.5 Train-Test Split Strategy .....                       | 16 |
| 4. INDIVIDUAL FORECASTING MODELS.....                     | 18 |
| 4.1 Overview .....                                        | 18 |
| 4.2 Gradient Boosting Regressor (GBR) — scikit-learn..... | 18 |
| 4.2.1 Theoretical Background .....                        | 18 |
| 4.2.2 Hyperparameter Configuration.....                   | 19 |
| 4.2.3 Implementation.....                                 | 19 |

|                                                              |    |
|--------------------------------------------------------------|----|
| 4.3 XGBoost (Extreme Gradient Boosting) .....                | 20 |
| 4.3.1 Theoretical Background .....                           | 20 |
| 4.3.2 Advantages for Financial Forecasting .....             | 21 |
| 4.4 LightGBM (Light Gradient Boosting Machine) .....         | 22 |
| 4.4.1 Theoretical Background .....                           | 22 |
| 4.4.2 Leaf-wise Tree Growth.....                             | 22 |
| 4.5 CatBoost.....                                            | 23 |
| 4.5.1 Theoretical Background .....                           | 23 |
| 4.5.2 Performance Characteristics .....                      | 24 |
| 4.6 Recursive Multi-Step Forecast .....                      | 25 |
| 5. ENSEMBLE MODEL AND COMBINATION STRATEGY .....             | 27 |
| 5.1 Motivation for Ensemble Methods.....                     | 27 |
| Bias–Variance Tradeoff.....                                  | 27 |
| 5.2 EnsembleModel Class Architecture .....                   | 27 |
| 5.3 Inverse-MAE Weighting Strategy.....                      | 29 |
| 5.4 Ensemble Training and Backtest.....                      | 30 |
| 5.5 Ensemble Forecast Generation.....                        | 31 |
| 6. PROJECT SDLC AND REQUIREMENTS .....                       | 32 |
| 6.1 Software Development Life Cycle.....                     | 32 |
| Phase 1: Requirements Analysis and System Design.....        | 32 |
| Phase 2: Data Pipeline and Feature Engineering.....          | 32 |
| Phase 3: Individual Model Implementation.....                | 32 |
| Phase 4: Ensemble Integration and Dashboard Development..... | 32 |
| Phase 5: Backtesting and Report Generation .....             | 32 |
| 6.2 Functional Requirements.....                             | 33 |
| 6.2.1 Data Management Requirements.....                      | 33 |
| 6.2.2 Forecasting Requirements .....                         | 33 |
| 6.2.3 Dashboard Requirements.....                            | 33 |
| 6.2.4 Reporting Requirements .....                           | 34 |
| 6.3 Non-Functional Requirements .....                        | 34 |
| 6.3.1 Performance.....                                       | 34 |
| 6.3.2 Accuracy .....                                         | 34 |
| 6.3.3 Reliability and Availability .....                     | 34 |
| 6.3.4 Maintainability and Extensibility .....                | 35 |

|                                                       |    |
|-------------------------------------------------------|----|
| 6.3.5 Security .....                                  | 35 |
| 7. SYSTEM IMPLEMENTATION .....                        | 36 |
| 7.1 Directory Structure and Module Organization.....  | 36 |
| 7.2 Configuration Module (config.py) .....            | 37 |
| 7.3 Streamlit Dashboard Implementation (app.py) ..... | 37 |
| 7.3.1 Page Configuration and Styling.....             | 37 |
| 7.3.2 Caching Strategy.....                           | 38 |
| 7.3.3 Eight-Tab Dashboard.....                        | 38 |
| 7.4 Evaluation Metrics Implementation .....           | 39 |
| 7.5 Technical Analysis Implementation.....            | 40 |
| 7.6 Risk Metrics Implementation .....                 | 40 |
| 7.7 News Sentiment Analysis.....                      | 41 |
| 7.8 Batch Backtest Report Generation .....            | 42 |
| 8. SYSTEM DESIGN DIAGRAMS.....                        | 43 |
| 8.1 Application Flow Diagram.....                     | 43 |
| 8.2 WorkFlow Diagram.....                             | 44 |
| 9. RESULTS AND BACKTESTING.....                       | 45 |
| 9.1 Backtest Methodology.....                         | 45 |
| 9.2 Ensemble vs. Individual Model Performance .....   | 45 |
| 9.3 Performance Across Asset Categories .....         | 46 |
| 9.3.1 Commodity Futures .....                         | 46 |
| 9.3.2 US NASDAQ Stocks.....                           | 46 |
| 9.3.3 Indian BSE Sensex Stocks.....                   | 46 |
| 9.4 Impact of History Window.....                     | 46 |
| 9.5 Feature Importance Analysis.....                  | 47 |
| 10. TECHNOLOGIES AND LIBRARIES USED .....             | 48 |
| 10.1 Programming Language: Python 3.10+ .....         | 48 |
| 10.2 Streamlit .....                                  | 48 |
| 10.3 yfinance .....                                   | 48 |
| 10.4 scikit-learn.....                                | 49 |
| 10.5 XGBoost.....                                     | 49 |
| 10.6 LightGBM.....                                    | 49 |
| 10.7 CatBoost.....                                    | 49 |
| 10.8 Pandas and NumPy.....                            | 50 |

|                                     |    |
|-------------------------------------|----|
| 10.9 Plotly .....                   | 50 |
| 10.10 PyArrow and openpyxl.....     | 50 |
| 10.11 Development Environment ..... | 51 |
| 11. VISUAL DEMONSTRATIONS .....     | 52 |
| 11.1 Dashboard Sidebar .....        | 52 |
| 11.2 Overview and Forecast Tab..... | 53 |
| 11.3 Backtest Tab.....              | 54 |
| 11.4 Insights Tab .....             | 55 |
| 11.5 Technical Analysis Tab.....    | 56 |
| 11.6 Risk Metrics Tab .....         | 57 |
| 11.7 News Sentiment Tab .....       | 58 |
| 11.8 Trading Signals Tab .....      | 59 |
| 11.9 Model Comparison.....          | 61 |
| 11.10 Batch Backtest Report.....    | 61 |
| 12. CONCLUSION.....                 | 63 |
| 12.1 Summary of Achievements .....  | 63 |
| 12.2 Discussion of Results .....    | 63 |
| 12.3 Limitations .....              | 64 |
| 12.4 Future Work .....              | 65 |
| 13. LITERATURE REFERENCES.....      | 66 |

---



## LIST OF FIGURES

| Figure No. | Description                                        | Page No. |
|------------|----------------------------------------------------|----------|
| Figure 1   | Application Flow Diagram                           | 41       |
| Figure 2   | Workflow Diagram                                   | 42       |
| Figure 3   | Dashboard Sidebar                                  | 50       |
| Figure 4   | Histoy and Recursive Multiday Forecast             | 51       |
| Figure 5   | Top Important Features                             | 51       |
| Figure 6   | Backtest Tab: Actual vs Predicted                  | 52       |
| Figure 7   | Insights                                           | 53       |
| Figure 8   | Technical Analysis                                 | 54       |
| Figure 9   | Moving Averages and Volatility                     | 54       |
| Figure 10  | Risk Metrics Tab — Drawdown Chart                  | 55       |
| Figure 11  | News Sentiment Analysis Flow                       | 56       |
| Figure 12  | Trading Signals                                    | 57       |
| Figure 13  | Performance Metrics Comparison of all 4 Models     | 58       |
| Figure 14  | Model Weights & Metrics                            | 58       |
| Figure 15  | Backtest Report Sample Output — MAPE across Assets | 59       |

## ABBREVIATIONS

| Abbreviation | Full Form                                                       |
|--------------|-----------------------------------------------------------------|
| ML           | Machine Learning                                                |
| GBR          | Gradient Boosting Regressor                                     |
| XGB          | Extreme Gradient Boosting                                       |
| LGBM         | Light Gradient Boosting Machine                                 |
| MAE          | Mean Absolute Error                                             |
| RMSE         | Root Mean Squared Error                                         |
| MAPE         | Mean Absolute Percentage Error                                  |
| VaR          | Value at Risk                                                   |
| CVaR         | Conditional Value at Risk                                       |
| SMA          | Simple Moving Average                                           |
| API          | Application Programming Interface                               |
| OHLCV        | Open, High, Low, Close, Volume                                  |
| GUI          | Graphical User Interface                                        |
| DFD          | Data Flow Diagram                                               |
| SDLC         | Software Development Life Cycle                                 |
| NSE          | National Stock Exchange                                         |
| BSE          | Bombay Stock Exchange                                           |
| NASDAQ       | National Association of Securities Dealers Automated Quotations |

|      |                                              |
|------|----------------------------------------------|
| CART | Classification and Regression Trees          |
| GBDT | Gradient Boosted Decision Trees              |
| OHE  | Ordered Target Statistics / One-Hot Encoding |
| CSV  | Comma Separated Values                       |
| TOC  | Table of Contents                            |
| UI   | User Interface                               |
| MSE  | Mean Squared Error                           |

# ABSTRACT

In an era of rapidly evolving global financial markets and information overload, making accurate predictions of stock and commodity prices is of immense practical and economic value. Traditional methods of financial analysis — whether fundamental or technical — are often subjective, time-consuming, and limited in their ability to capture complex non-linear patterns present in financial time-series data. Machine learning and ensemble methods offer a powerful alternative by learning from historical price data automatically and producing objective, data-driven forecasts.

This project presents the design and implementation of a full-stack Stocks and Commodities Price Forecasting Engine — an end-to-end machine learning application that downloads historical price data from Yahoo Finance, engineers a rich set of predictive features, trains an ensemble of four state-of-the-art gradient-boosted regression models, and presents actionable forecasts through an interactive web-based dashboard.

The core of the forecasting engine is a heterogeneous ensemble that combines four individually trained models: Gradient Boosting Regressor (GBR) from scikit-learn, XGBoost (Extreme Gradient Boosting), LightGBM (Light Gradient Boosting Machine), and CatBoost. Rather than using naive equal-weighting, the ensemble adopts an Inverse Mean Absolute Error (Inverse-MAE) weighting strategy — models that perform better on a held-out backtest period automatically receive proportionally higher influence in the final prediction. This self-tuning mechanism makes the ensemble adaptive to the characteristics of each individual asset without requiring manual hyperparameter tuning of the weights.

The feature engineering pipeline constructs lag features (1-7 days of historical closing prices), rolling statistical features (5-day and 10-day moving averages and standard deviations), and one-day return percentage — providing the models with both short-term momentum and medium-term trend

signals. A strict chronological train-test split (80% training, 20% testing) is enforced to prevent data leakage, ensuring that all reported accuracy metrics are genuine out-of-sample estimates.

The system covers a diverse asset universe of 40 assets: 5 commodity futures (WTI Crude Oil, Natural Gas, Copper, Coffee, Sugar), 12 top NASDAQ-listed US equities, and 23 BSE Sensex-listed Indian stocks. Forecasts are generated using a recursive multi-step prediction strategy for horizons of 5-30 trading days. The Streamlit-powered dashboard presents results through eight analytical tabs: forecast visualization, backtesting, model insights, descriptive statistics, live news sentiment, rule-based trading signals, technical analysis (SMA, momentum), and risk metrics (VaR, CVaR, Sharpe ratio, Max Drawdown).

Backtesting across all assets and multiple history windows demonstrates that the ensemble consistently outperforms each individual constituent model in terms of MAE, RMSE, and MAPE, validating the effectiveness of the ensemble approach for financial time-series forecasting. The system represents a comprehensive, extensible, and practically deployable solution for short-to-medium-term price forecasting in financial markets.

# 1. INTRODUCTION

## 1.1 Context and Background

Financial markets are among the most complex, dynamic, and information-rich environments in the modern world. Every trading day, millions of transactions occur across stock exchanges, commodity markets, and derivatives platforms worldwide. The prices of financial instruments — stocks, futures, indices, and commodities — are influenced by a bewildering array of factors including macroeconomic indicators, geopolitical events, company earnings, investor sentiment, supply-demand dynamics, and even social media trends. The ability to accurately forecast these prices, even a few days in advance, can yield substantial economic benefits for investors, traders, and portfolio managers.

Traditional approaches to financial forecasting broadly fall into two schools: fundamental analysis, which examines the intrinsic value of an asset based on economic and financial data, and technical analysis, which relies on historical price patterns and statistical indicators to predict future movements. While both approaches have their merits, they are inherently limited by human cognitive bandwidth, the complexity of the relationships involved, and the sheer volume of data that must be processed in real time.

The emergence of machine learning (ML) and artificial intelligence has fundamentally transformed the landscape of quantitative finance. ML algorithms are capable of identifying complex non-linear patterns in high-dimensional datasets that would be impossible for human analysts to detect manually. In particular, ensemble methods — which combine the predictions of multiple individual models — have emerged as one of the most powerful and reliable approaches for tabular time-series prediction tasks. Gradient boosting algorithms, in particular, have consistently dominated competitive machine learning benchmarks and have been widely adopted in industry for financial modeling.

## 1.2 Problem Statement

Despite the availability of powerful ML tools and financial data APIs, there remains a gap between academic research in financial ML and accessible, production-ready forecasting systems that non-expert users can interact with. Most existing systems are either overly simplistic (using single models) or overly complex (requiring specialized quant finance expertise). Furthermore, few systems combine real-time data ingestion, robust ensemble forecasting, backtesting, risk analysis, news sentiment, and trading signal generation in a unified, user-friendly interface.

This project addresses this gap by building a comprehensive, end-to-end Stocks and Commodities Price Forecasting Engine that:

- Automatically downloads and caches historical price data for a diverse universe of 40 global assets.
- Constructs a rich feature matrix from price history using lag and rolling statistical features.
- Trains four state-of-the-art gradient boosting models and combines them using an adaptive inverse-MAE ensemble weighting strategy.
- Performs rigorous out-of-sample backtesting with MAE, RMSE, and MAPE metrics.
- Generates recursive multi-step price forecasts for user-selected horizons.
- Presents all results through an interactive, eight-tab Streamlit web dashboard.
- Incorporates technical analysis, risk metrics, news sentiment, and rule-based trading signals.

## 1.3 Motivation

The motivation for this project arises from multiple sources. First, the democratization of financial data through APIs like Yahoo Finance has made high-quality price data freely accessible, removing a key barrier to building ML-based forecasting systems. Second, the maturation of gradient boosting libraries (XGBoost, LightGBM, CatBoost) has made it possible to train highly accurate models with relatively modest computational resources. Third, the availability of

Streamlit as a rapid web app development framework makes it feasible to build interactive dashboards without full-stack web development expertise.

From an academic standpoint, this project provides an opportunity to apply, compare, and synthesize a range of advanced machine learning techniques — gradient boosting, ensemble methods, feature engineering, time-series cross-validation, recursive forecasting — in a real-world domain with genuine practical significance. The project also addresses important concerns around model interpretability (through feature importance analysis), generalization (through strict out-of-sample backtesting), and responsible deployment (through prominent risk disclaimers).

## **1.4 Types of Forecasting Approaches**

### **1.4.1 Statistical Methods**

Classical statistical models such as ARIMA (AutoRegressive Integrated Moving Average), SARIMA, and GARCH have long been the standard tools for financial time-series forecasting. These models are based on explicit mathematical formulations and make strong assumptions about the stationarity and linearity of the data. While they are interpretable and computationally efficient, they struggle to capture complex non-linear relationships and interactions between multiple predictors.

### **1.4.2 Machine Learning Methods**

Machine learning approaches treat price forecasting as a supervised regression problem: given a vector of features derived from historical price data, predict the next-day closing price. Algorithms such as support vector regression, random forests, and gradient boosted trees can capture complex non-linear patterns without making strong distributional assumptions. The key challenge is feature engineering — constructing informative predictors from raw price data — and avoiding data leakage through proper temporal splitting.

### **1.4.3 Deep Learning Methods**

Recurrent Neural Networks (RNNs), Long Short-Term Memory (LSTM) networks, and Transformer-based models have shown promise for financial time-series due to their ability to



model temporal dependencies. However, they typically require large amounts of training data, significant computational resources, and careful regularization to avoid overfitting. For the moderate-sized datasets available through Yahoo Finance, gradient boosting models often match or outperform deep learning approaches while being far simpler to train, interpret, and deploy.

#### **1.4.4 Ensemble Methods**

Ensemble methods combine the predictions of multiple base models to produce a final prediction that is typically more accurate and robust than any individual model. The intuition is that different models capture different aspects of the data, and their errors are partially uncorrelated, so averaging reduces overall prediction error. The two main paradigms are bagging (training models independently on bootstrap samples, as in Random Forests) and boosting (training models sequentially where each model corrects the errors of its predecessor). This project employs boosting-based models as base learners and combines them through weighted averaging — a form of stacking.

### **1.5 Scope and Objectives of the Project**

The primary objective of this project is to design, implement, and evaluate a full-stack machine learning application for financial price forecasting. The specific objectives are:

1. To build an automated data pipeline that downloads, cleans, and caches daily OHLCV price data from Yahoo Finance for a diverse universe of commodities, US stocks, and Indian stocks.
2. To implement a feature engineering module that constructs lag features and rolling statistics from raw price data.
3. To train and evaluate four gradient boosting models (GBR, XGBoost, LightGBM, CatBoost) individually using strict chronological train-test splits.
4. To implement an adaptive ensemble model that combines the four individual models using inverse-MAE weighting.
5. To generate recursive multi-step price forecasts and evaluate their accuracy through backtesting.

6. To develop an interactive Streamlit dashboard with eight analytical tabs covering forecasts, backtesting, statistics, news, trading signals, technical analysis, and risk metrics.
7. To generate a comprehensive batch backtest report covering all 40 assets across multiple history windows.

## 2. LITERATURE SURVEY

### 2.1 Overview

The application of machine learning to financial time-series forecasting has been an active area of research for over three decades. Early work focused on applying neural networks to stock price prediction, with mixed results due to overfitting and the non-stationarity of financial data. The 2010s saw a resurgence of interest driven by the availability of large datasets, improved regularization techniques, and the development of powerful gradient boosting algorithms. This section surveys the key developments in the field that are most relevant to this project.

### 2.2 Gradient Boosting for Financial Forecasting

Gradient boosting machines, introduced by Friedman (2001), have become one of the dominant approaches for tabular data prediction tasks. The key innovation is the sequential training of weak learners (decision trees) where each tree is trained to minimize the residuals of the previous ensemble. Friedman showed that this procedure is equivalent to gradient descent in function space, and that using shallow trees as base learners provides a natural regularization.

Chen and Guestrin (2016) introduced XGBoost, which extended gradient boosting with second-order Taylor approximations of the loss function, column subsampling, and built-in L1/L2 regularization. XGBoost quickly became the dominant algorithm in machine learning competitions and has been widely applied to financial prediction tasks. Studies have shown that XGBoost achieves competitive accuracy for stock return prediction when combined with appropriate feature engineering.

Ke et al. (2017) introduced LightGBM, which addressed the computational bottleneck of gradient boosting by introducing two novel techniques: Gradient-based One-Side Sampling (GOSS) and Exclusive Feature Bundling (EFB). GOSS selectively retains samples with large gradients (informative samples) while randomly sampling those with small gradients, dramatically reducing the effective dataset size without sacrificing accuracy. EFB bundles mutually exclusive sparse

features, reducing the feature dimensionality. These innovations make LightGBM significantly faster than XGBoost on large datasets.

Prokhorenkova et al. (2018) introduced CatBoost, developed by Yandex, which addressed a critical source of bias in gradient boosting — the use of target statistics for categorical feature encoding, which can lead to data leakage. CatBoost introduces ordered boosting, which computes target statistics using only the data preceding each training sample in a randomly permuted order, eliminating this bias. CatBoost has shown strong performance on datasets with categorical features.

## **2.3 Ensemble Methods for Price Prediction**

The superiority of ensemble methods over individual models in financial prediction has been well documented in the literature. Dietterich (2000) provided a theoretical foundation for ensemble methods, showing that they reduce variance without increasing bias when the base learner predictions are sufficiently uncorrelated. In the financial domain, where different models may capture different aspects of market dynamics (momentum, mean-reversion, volatility regimes), ensembling is particularly valuable.

Several studies have demonstrated the effectiveness of combining gradient boosting models for stock prediction. Khaidem et al. (2016) showed that random forests and gradient boosting outperform logistic regression and SVM for stock return classification. Fischer and Krauss (2018) found that LSTM networks outperform standard ML approaches on long-horizon predictions, but gradient boosting remains competitive for short-horizon (1-5 day) forecasts.

Inverse-MAE weighting, used in this project, is a form of performance-based ensemble weighting that has been studied in the context of forecast combination. The optimal combination of forecasts was first studied by Bates and Granger (1969), who showed that the combined forecast can outperform even the best individual forecast. Recent work has extended this to time-varying weights that adapt to changing model performance, providing additional robustness.

## 2.4 Feature Engineering for Financial Time-Series

Feature engineering is widely recognized as one of the most critical components of a successful ML pipeline for financial data. Raw price data alone is highly non-stationary, and models trained directly on prices often generalize poorly. The most common approach is to construct features that capture different aspects of price dynamics:

- Lag features (prices and returns at  $t-1$ ,  $t-2$ , ...,  $t-n$ ) provide the model with information about recent price history.
- Rolling statistics (moving averages, moving standard deviations) capture medium-term trends and volatility.
- Momentum indicators (rate of change, RSI, MACD) capture trending behavior.
- Volume-based features capture information about market participation.

Studies have consistently found that lag features and rolling statistics are among the most informative predictors for short-horizon price forecasting. The specific lag lengths and window sizes are typically selected based on domain knowledge and empirical validation.

## 2.5 Backtesting and Model Evaluation

A critical challenge in financial ML is the proper evaluation of out-of-sample performance. Naive random cross-validation is inappropriate for time-series data because it allows future information to leak into the training set. The correct approach is walk-forward validation (also called time-series cross-validation), where the training set always precedes the test set chronologically.

The selection of appropriate evaluation metrics is also important. Mean Absolute Error (MAE) provides an easily interpretable measure of average prediction error in price units. Root Mean Squared Error (RMSE) penalizes large errors more heavily and is more sensitive to outliers. Mean Absolute Percentage Error (MAPE) normalizes the error by the actual price, making it comparable across assets with different price scales.

## 2.6 Relevant Prior Work and Gaps

While there is a substantial body of literature on individual components — gradient boosting for finance, feature engineering for time-series, ensemble combination strategies — there are relatively few publicly available, end-to-end systems that integrate all these components in a production-ready, user-friendly application. Most research prototypes focus on a narrow set of assets (often a single stock index), use a single model, and present results only in static academic format.

This project addresses these gaps by building a unified, interactive system that covers a diverse asset universe, trains and compares four state-of-the-art models, combines them adaptively, and presents results through a professional web dashboard with multiple analytical perspectives.

## 3. METHODOLOGY

### 3.1 System Architecture Overview

The Stocks and Commodities Price Forecasting Engine is built around a modular, five-stage pipeline architecture that cleanly separates concerns: data ingestion, feature engineering, model training, ensemble prediction, and presentation. Each stage is implemented as a separate Python module, making the system extensible and maintainable.

The overall data flow is as follows: raw daily OHLCV price data is downloaded from Yahoo Finance and cached locally as Parquet files. The feature engineering module transforms the raw close price series into a supervised ML feature matrix. Four gradient boosting models are trained on this matrix using a strict chronological 80/20 train-test split. Each model is evaluated on the held-out test set to compute backtest metrics and derive ensemble weights. The full ensemble model is then used to generate recursive multi-step forecasts. Finally, the Streamlit dashboard integrates all results and additional analyses (news sentiment, technical indicators, risk metrics) into an interactive web application.

### 3.2 Asset Universe

The system covers three categories of assets, all accessed through Yahoo Finance tickers:

| Category          | Count | Examples                                                                             |
|-------------------|-------|--------------------------------------------------------------------------------------|
| Commodity Futures | 13    | WTI Crude Oil (CL=F), Natural Gas (NG=F), Copper (HG=F), Coffee (KC=F), Sugar (SB=F) |
| US NASDAQ Stocks  | 23    | AAPL, MSFT, AMZN, TSLA, META, NFLX, AMD, QQQ, COST, PEP, WMT, ABNB                   |

|                   |    |                                                                                        |
|-------------------|----|----------------------------------------------------------------------------------------|
| Indian BSE Sensex | 35 | TCS.NS, RELIANCE.NS,<br>HDFCBANK.NS, INFY.NS,<br>WIPRO.NS, ICICIBANK.NS,<br>and others |
|-------------------|----|----------------------------------------------------------------------------------------|

### 3.3 Data Collection and Caching (data\_loader.py)

Data collection is handled by the `load_commodity_history()` function in `src/data_loader.py`. This function implements a two-level caching strategy to minimize API calls and ensure fast response times. The primary cache is a Parquet file stored on disk, keyed by a deterministic MD5 hash of the ticker, period, and interval parameters. If the cache file exists and contains data, it is returned immediately without making any network calls.

If no cached data exists, the function calls `yf.download()` from the `yfinance` library to fetch historical daily bars from Yahoo Finance. The raw data is then processed through a validation and normalization pipeline:

8. Multi-level column indices (sometimes produced by `yfinance`) are flattened.
9. All column names are normalized to lowercase.
10. The presence of required columns (open, high, low, close, volume) is verified.
11. The DataFrame is restricted to these five columns.
12. Timezone information is stripped from the DatetimeIndex for consistency.
13. The index is sorted chronologically and fully-empty rows are dropped.
14. The cleaned DataFrame is saved as a Parquet file for future reuse.



The following code snippet illustrates the core data loading logic:

```
def load_commodity_history(ticker, period='5y', interval='1d', use_cache=True):  
    path = _cache_path(ticker, period, interval)  
    if use_cache and path.exists():  
        df = pd.read_parquet(path)  
        if not df.empty: return df  
  
    raw = yf.download(ticker, period=period, interval=interval,  
                      progress=False, auto_adjust=True)  
    if raw.empty:  
        raise ValueError(f'No data returned for {ticker}.')  
  
    df = raw.copy()  
    if isinstance(df.columns, pd.MultiIndex):  
        df.columns = [c[0] for c in df.columns]  
    df.columns = [str(c).strip().lower() for c in df.columns]  
    df = df[['open','high','low','close','volume']].dropna(how='all')  
    df.index = pd.to_datetime(df.index).tz_localize(None)  
    df = df.sort_index()  
    if use_cache: df.to_parquet(path)  
    return df
```

### 3.4 Feature Engineering (features.py)

The feature engineering module transforms a univariate series of daily closing prices into a rich, multi-dimensional feature matrix suitable for supervised ML training. The target variable is the next-day closing price ( $y = \text{close.shift}(-1)$ ). Features are constructed as follows:

| Feature Name    | Formula                                                        | Description                                    |
|-----------------|----------------------------------------------------------------|------------------------------------------------|
| close           | raw price                                                      | Current day's closing price (context)          |
| lag_1 ... lag_7 | <code>close.shift(1) ...</code><br><code>close.shift(7)</code> | Closing prices 1-7 trading days ago            |
| ret_1           | <code>close.pct_change(1)</code>                               | 1-day percentage return (momentum)             |
| roll_mean_5     | <code>close.rolling(5).mean()</code>                           | 5-day simple moving average                    |
| roll_mean_10    | <code>close.rolling(10).mean()</code>                          | 10-day simple moving average                   |
| roll_std_5      | <code>close.rolling(5).std()</code>                            | 5-day rolling standard deviation (volatility)  |
| roll_std_10     | <code>close.rolling(10).std()</code>                           | 10-day rolling standard deviation              |
| y (target)      | <code>close.shift(-1)</code>                                   | Next-day closing price — the prediction target |

Rows containing any NaN values (arising from the initial lag computation or rolling window warm-up period) are dropped before the matrix is returned. No imputation is performed — this is critical for time-series correctness. The `feature_columns()` function returns all column names except 'close' itself, ensuring that the raw closing price is not used as a direct predictor (preventing a trivially dominant feature).

```

def build_feature_matrix(close):
    s = close.astype(float)
    out = pd.DataFrame(index=s.index)
    out['close'] = s
    for lag in LAG_DAYS: # [1,2,3,4,5,6,7]
        out[f'lag_{lag}'] = s.shift(lag)
    out['ret_1'] = s.pct_change(1)
    for w in ROLL_WINDOWS: # (5, 10)
        out[f'roll_mean_{w}'] = s.rolling(w).mean()
        out[f'roll_std_{w}'] = s.rolling(w).std()
    y = s.shift(-1)
    mask = out.notna().all(axis=1) & y.notna()
    return out.loc[mask].copy(), y.loc[mask].copy()

```

### 3.5 Train-Test Split Strategy

A strict chronological train-test split is enforced throughout the system. The last TEST\_SIZE\_FRAC (20%) of time-ordered rows are held out as the test set; the preceding 80% constitute the training set. No random shuffling is performed. This preserves the temporal ordering of financial data and prevents data leakage — a critical requirement for honest out-of-sample evaluation of time-series models.

The split boundaries are computed as follows:

```

n = len(X)

split = int(n * (1 - TEST_SIZE_FRAC)) # TEST_SIZE_FRAC = 0.2

split = max(split, n // 2) # at least 50% training

split = min(split, n - 10) # at least 10 test samples

```

```
X_train, X_test = X.iloc[:split], X.iloc[split:]  
y_train, y_test = y.iloc[:split], y.iloc[split:]
```

After evaluating on the test set, each model is refitted on the complete feature matrix (100% of data) before generating the final deployment forecast. This Train-Evaluate-Refit pattern is standard practice: honest metrics are measured using held-out data, and the final deployed model is trained on all available history for maximum predictive power.

## 4. INDIVIDUAL FORECASTING MODELS

### 4.1 Overview

Four gradient-boosted tree regression models are trained independently on the same feature matrix. Despite being different implementations from different libraries, all four models share the same hyperparameter configuration and API contract — enabling clean comparison and ensemble combination. Each model file exposes the same two functions: `train_and_backtest()` and `attach_forecast()`, and returns the same `BacktestResult` and `TrainForecastResult` dataclass types.

### 4.2 Gradient Boosting Regressor (GBR) — scikit-learn

#### 4.2.1 Theoretical Background

Gradient Boosting Machines (GBM), introduced by Jerome Friedman in 2001, are ensemble methods that build an additive model in a forward stage-wise fashion. At each stage, a decision tree is fitted to the negative gradient (pseudo-residuals) of the loss function with respect to the current ensemble's predictions. For regression with squared-error loss, the pseudo-residuals are simply the residuals from the current prediction.

The ensemble prediction at iteration  $M$  is:

$$F_M(x) = F_0(x) + \sum_{m=1}^M v * h_m(x)$$

where:

$F_0(x)$  = initial prediction (usually the mean of  $y$ )

$h_m(x)$  =  $m$ -th decision tree fitted to pseudo-residuals

$v$  = learning rate (shrinkage parameter)

$M$  = number of trees (`n_estimators`)

The learning rate  $\nu$  acts as a regularizer: smaller values require more trees to achieve the same training error but typically yield better generalization. The use of shallow trees (small `max_depth`) provides another form of regularization by restricting the complexity of each individual learner.

### 4.2.2 Hyperparameter Configuration

| Parameter                  | Value | Rationale                                                                                            |
|----------------------------|-------|------------------------------------------------------------------------------------------------------|
| <code>n_estimators</code>  | 200   | Sufficient trees for good generalization; tested against alternatives (100, 300, 500)                |
| <code>max_depth</code>     | 4     | Shallow trees prevent overfitting on financial time-series with limited signal-to-noise ratio        |
| <code>learning_rate</code> | 0.05  | Conservative step size; 0.05 is well-supported by literature for gradient boosting on financial data |
| <code>subsample</code>     | 0.9   | Row subsampling adds stochasticity, reducing variance and overfitting; 0.9 retains most data         |
| <code>random_state</code>  | 42    | Fixed seed for complete reproducibility across runs                                                  |

### 4.2.3 Implementation

```
from sklearn.ensemble import GradientBoostingRegressor

model = GradientBoostingRegressor(
    n_estimators=200,
    max_depth=4,
    learning_rate=0.05,
    subsample=0.9,
    random_state=42,
```

```
)  
model.fit(X_train[feats], y_train)  
y_pred = model.predict(X_test[feats])
```

The scikit-learn GBR is the baseline model in the ensemble. It uses the standard Friedman gradient boosting algorithm with squared error loss. While it is slower than XGBoost and LightGBM for large datasets, it provides a reliable, well-tested reference implementation.

## 4.3 XGBoost (Extreme Gradient Boosting)

### 4.3.1 Theoretical Background

XGBoost, developed by Tianqi Chen at the University of Washington and open-sourced in 2014, extends the standard gradient boosting framework with several key innovations:

- **Second-Order Optimization:** XGBoost uses both the first-order (gradient) and second-order (Hessian) derivatives of the loss function to compute more accurate leaf weights, resulting in faster and more stable convergence.
- **Regularization:** XGBoost adds explicit L1 (Lasso) and L2 (Ridge) regularization terms to the objective function, directly penalizing model complexity and reducing overfitting.
- **Column (Feature) Subsampling:** By randomly sampling a subset of features at each tree node, XGBoost introduces additional randomness that improves generalization and speeds up training.
- **Sparsity-Aware Split Finding:** XGBoost has a built-in mechanism for handling missing values efficiently.
- **Approximate Tree Learning:** XGBoost uses quantile sketches to approximate the optimal split points, enabling efficient processing of large datasets.

The XGBoost objective function for regression is:

$$\text{Obj} = \sum_i L(y_i, \hat{y}_i) + \sum_k \Omega(f_k)$$

where:

$L(y_i, \hat{y}_i)$  = loss function (squared error for regression)

$\Omega(f_k)$  = regularization term =  $\gamma T + \frac{1}{2} \lambda \|w\|^2$

$T$  = number of leaves in tree  $k$

$w$  = leaf weight vector

$\gamma, \lambda$  = regularization hyperparameters

### 4.3.2 Advantages for Financial Forecasting

XGBoost's combination of second-order optimization and regularization makes it particularly well-suited for financial time-series where: (1) the signal-to-noise ratio is low, requiring strong regularization to avoid overfitting; (2) the optimal split points are not uniformly distributed across feature values, benefiting from the flexible approximate split-finding algorithm; and (3) missing data from trading holidays or data gaps must be handled gracefully.

```
from xgboost import XGBRegressor
```

```
model = XGBRegressor(
```

```
    n_estimators=200,
```

```
    max_depth=4,
```

```
    learning_rate=0.05,
```

```
    subsample=0.9,
```

```
    random_state=42,
```

```
    verbosity=0,
```



```
)  
model.fit(X_train[feats], y_train)
```

## 4.4 LightGBM (Light Gradient Boosting Machine)

### 4.4.1 Theoretical Background

LightGBM was developed by Microsoft Research and introduced in 2017 specifically to address the computational challenges of gradient boosting on large-scale datasets. Its two key innovations — Gradient-based One-Side Sampling (GOSS) and Exclusive Feature Bundling (EFB) — make it significantly faster than XGBoost without sacrificing accuracy.

Gradient-based One-Side Sampling (GOSS) is based on the observation that samples with small gradient magnitudes have already been well-fitted by the current model and contribute less to the overall information gain. GOSS retains all high-gradient samples (those with gradients above a threshold  $a$ ) and randomly samples a fraction  $b$  from the remaining low-gradient samples. The low-gradient samples are then upweighted by a factor  $(1-a)/b$  to compensate for the reduced sampling. This reduces the effective training set size while preserving the most informative samples.

Exclusive Feature Bundling (EFB) exploits the sparsity of feature matrices. In many real-world datasets, features are mutually exclusive — they rarely take non-zero values simultaneously. EFB bundles such features into a single combined feature, reducing the feature dimensionality and speeding up split finding. For the feature matrix in this project (which has no sparse features), EFB provides less benefit, but GOSS still accelerates training substantially.

### 4.4.2 Leaf-wise Tree Growth

A critical architectural difference between LightGBM and other gradient boosting implementations is the tree growth strategy. Standard gradient boosting uses level-wise (breadth-first) growth, expanding all leaves at the same depth. LightGBM uses leaf-wise (best-first) growth, always expanding the leaf with the maximum loss reduction regardless of depth. This produces asymmetric trees that can more accurately model complex non-linear relationships, at the cost of a higher risk of overfitting on small datasets. The `max_depth` parameter in LightGBM controls the maximum depth to prevent excessive tree growth.

```
import lightgbm as lgb

model = lgb.LGBMRegressor(
    n_estimators=200,
    max_depth=4,
    learning_rate=0.05,
    subsample=0.9,
    random_state=42,
    verbose=-1,      # suppress training output
    min_child_samples=1, # allow small leaf nodes
)
model.fit(X_train[feats], y_train)
```

## 4.5 CatBoost

### 4.5.1 Theoretical Background

CatBoost was developed by Yandex and released in 2018. Its name reflects its two primary distinguishing features: Categorical feature support and Boosting. While CatBoost's approach to categorical features is not directly relevant here (all features are numerical), its other innovation — Ordered Boosting — addresses a fundamental source of bias in standard gradient boosting that is directly relevant to any dataset.

**The Problem:** In standard gradient boosting, the pseudo-residuals (targets for each tree) are computed using the current ensemble's predictions. However, the ensemble was trained on the same data used to compute the pseudo-residuals, leading to a conditional shift — the targets are biased toward the predictions of the model on its own training data. This is a subtle but real source of overfitting.

Ordered Boosting Solution: CatBoost eliminates this bias by maintaining a random permutation of the training samples and computing the pseudo-residual for each sample using a model trained only on the samples that precede it in the permutation. This ensures that the targets for each sample are always computed using a model that has not seen that sample, eliminating the conditional shift. Additionally, CatBoost uses symmetric (oblivious) decision trees as base learners, where the same split criterion is applied at all nodes at the same depth. This constraint reduces the variance of the model and enables efficient CPU/GPU implementation.

### 4.5.2 Performance Characteristics

CatBoost generally achieves competitive accuracy compared to XGBoost and LightGBM, with the additional benefit of reduced overfitting due to Ordered Boosting. It is particularly strong on datasets where overfitting to the training set is a concern — as is the case with financial time-series where the signal-to-noise ratio is low.

```
from catboost import CatBoostRegressor

model = CatBoostRegressor(
    iterations=200,    # equivalent to n_estimators
    depth=4,
    learning_rate=0.05,
    subsample=0.9,
    random_state=42,
    verbose=False,    # suppress training output
)
model.fit(X_train[feats], y_train)
```

## 4.6 Recursive Multi-Step Forecast

All four models support recursive (auto-regressive) multi-step forecasting through the `recursive_forecast()` function. The procedure is identical across all four models:

1. Start with the complete historical close price series.
2. Build the full feature matrix from the extended series.
3. Extract the last row of the feature matrix (most recent features).
4. Generate the next-day close price prediction.
5. Append the predicted price to the extended series.
6. Repeat steps 2-5 for the specified forecast horizon.

Future dates are generated using `pd.bdate_range` to produce business days only (excluding weekends). This approach naturally propagates each day's prediction into the next day's feature computation, creating a self-referential forecast chain.

```
def recursive_forecast(close, model, feature_names, horizon_days):  
    extended = close.astype(float).copy()  
    last_date = extended.index[-1]  
    future_dates = pd.bdate_range(  
        start=last_date + pd.offsets.BDay(1),  
        periods=horizon_days  
    )  
    preds = []  
    for i in range(horizon_days):  
        X_full, _ = build_feature_matrix(extended)  
        if X_full.empty: break  
        row = X_full.iloc[[-1]][feature_names]  
        next_close = float(model.predict(row)[0])
```

```
preds.append(next_close)

extended = pd.concat([extended,
    pd.Series([next_close], index=[future_dates[i]])])

return future_dates[:len(preds)], np.array(preds)
```

An important limitation of recursive forecasting is error accumulation: each predicted value is used as input for the next prediction, so errors compound over the horizon. Accuracy is typically strongest for 1-3 day horizons and degrades progressively for longer horizons. The system explicitly communicates this limitation to users.

## 5. ENSEMBLE MODEL AND COMBINATION STRATEGY

### 5.1 Motivation for Ensemble Methods

The theoretical justification for ensemble methods is well-established in the machine learning literature. When individual models make predictions that are sufficiently uncorrelated with each other, combining their predictions through averaging reduces the variance of the ensemble without increasing bias. In financial forecasting, where different models may capture different market regimes, momentum patterns, and volatility structures, ensemble combination is particularly valuable.

The key insight is that no single model is universally optimal across all assets and market conditions. GBR may perform best on stable, trending assets; LightGBM may excel on volatile, high-frequency assets; XGBoost may dominate in regimes with strong momentum; CatBoost may be most robust to overfitting in noisy environments. By combining all four, the ensemble achieves a form of model diversification analogous to portfolio diversification in finance.

#### Bias–Variance Tradeoff

In supervised learning, prediction error can be decomposed into:

$$\text{Error} = \text{Bias}^2 + \text{Variance} + \text{Noise}$$

- **Bias** → systematic error (model too simple)
- **Variance** → sensitivity to training data (model too complex)
- **Noise** → irreducible randomness

### 5.2 EnsembleModel Class Architecture

The ensemble is implemented through the EnsembleModel class in src/model\_ensemble.py. This class wraps four individually trained model objects and exposes a unified interface identical to any single model. This design pattern — known as the Composite pattern in software engineering —

makes the ensemble transparent to the rest of the system: any component that interacts with a model object can interact with the ensemble without modification.

```
class EnsembleModel:

    def __init__(self, models, weights, feature_names):

        self.models = models      # dict: {'gbr': ..., 'lightgbm': ..., ...}

        self.weights = weights    # dict: {'gbr': 0.3, 'lightgbm': 0.25, ...}

        self.feature_names = feature_names


    def predict(self, X):

        predictions = {name: model.predict(X)

                        for name, model in self.models.items()}

        ensemble_pred = np.zeros_like(predictions['gbr'], dtype=float)

        for name, pred in predictions.items():

            ensemble_pred += self.weights[name] * pred

        return ensemble_pred


    @property
    def feature_importances_(self):

        importances = {name: model.feature_importances_

                        for name, model in self.models.items()

                        if hasattr(model, 'feature_importances_')}

        ensemble_imp = np.zeros_like(next(iter(importances.values())))

        for name, imp in importances.items():

            ensemble_imp += self.weights[name] * imp
```

```
return ensemble_imp
```

## 5.3 Inverse-MAE Weighting Strategy

The ensemble uses an Inverse Mean Absolute Error (Inverse-MAE) weighting strategy to assign influence to each model. This approach is both principled and adaptive: models that demonstrate lower prediction error on the held-out backtest period automatically receive higher weight in the ensemble, without requiring any manual tuning.

The weighting formula is:

$$\text{weight}_i = (1 / \text{MAE}_i) / \sum_j (1 / \text{MAE}_j)$$

Example: If MAE values are GBR=2.0, LGBM=1.5, CatBoost=1.8, XGB=1.6

$$\text{inv\_gbr} = 1/2.0 = 0.500$$

$$\text{inv\_lgbm} = 1/1.5 = 0.667$$

$$\text{inv\_catboost} = 1/1.8 = 0.556$$

$$\text{inv\_xgb} = 1/1.6 = 0.625$$

$$\text{total} = 2.347$$

$$\text{w\_gbr} = 0.500 / 2.347 = 0.213$$

$$\text{w\_lgbm} = 0.667 / 2.347 = 0.284$$

$$\text{w\_catboost} = 0.556 / 2.347 = 0.237$$

$$\text{w\_xgb} = 0.625 / 2.347 = 0.266$$

This weighting has several desirable properties: (1) it is non-negative, ensuring the ensemble prediction is a convex combination of individual predictions; (2) the weights sum to 1; (3) a model with zero MAE would receive infinite weight (but this never occurs in practice); (4) a small epsilon (1e-6) is used as the denominator to prevent division by zero.



```
def _compute_ensemble_weights(results):
    mae_values = {name: result.backtest.mae
                  for name, result in results.items()}

    inv_sum = sum(1.0/max(mae, 1e-6) for mae in mae_values.values())

    weights = {name: (1.0/max(mae, 1e-6)) / inv_sum
              for name, mae in mae_values.items()}

    return weights
```

## 5.4 Ensemble Training and Backtest

The `train_and_backtest()` function in `model_ensemble.py` orchestrates the complete ensemble training workflow:

1. Train all four individual models sequentially (GBR, LightGBM, CatBoost, XGBoost) by calling their respective `train_and_backtest()` functions.
2. Extract the per-model backtest MAE values from their held-out test predictions.
3. Compute inverse-MAE weights and print them to the console for transparency.
4. Compute the ensemble's backtest predictions by weighted-averaging the four models' test-set predictions (which all share the same test index, since they use the same feature matrix).
5. Compute ensemble-level MAE, RMSE, and MAPE on the blended test predictions.
6. Create an `EnsembleModel` object wrapping the four trained model objects (refitted on full data).
7. Run the ensemble on the full feature matrix to generate in-sample predictions for visualization.
8. Return a `TrainForecastResult` with the ensemble model, backtest metrics, weights, and individual model references.

An important observation is that the ensemble's backtest MAE is typically lower than the best individual model's MAE, confirming the value of combining diverse models.

## 5.5 Ensemble Forecast Generation

The `attach_forecast()` function generates the final ensemble forecast by calling each individual model's `attach_forecast()` function independently and then combining their forecast arrays using the same inverse-MAE weights:

```
def attach_forecast(result, horizon_days):

    # Generate independent forecasts from each model

    gbr_fc = attach_gbr(result.individual_models['gbr'], horizon_days)
    lgbm_fc = attach_lightgbm(result.individual_models['lightgbm'], horizon_days)
    cat_fc = attach_catboost(result.individual_models['catboost'], horizon_days)
    xgb_fc = attach_xgb(result.individual_models['xgb'], horizon_days)

    # Weighted average of forecast arrays

    ensemble_forecast = (
        result.weights['gbr'] * gbr_fc.forecast_close +
        result.weights['lightgbm'] * lgbm_fc.forecast_close +
        result.weights['catboost'] * cat_fc.forecast_close +
        result.weights['xgb'] * xgb_fc.forecast_close
    )

    return TrainForecastResult(..., forecast_close=ensemble_forecast)
```

## **6. PROJECT SDLC AND REQUIREMENTS**

### **6.1 Software Development Life Cycle**

The project followed an iterative development model, structured into five phases. This approach allowed for incremental refinement of both the ML pipeline and the user interface, with each iteration producing a testable, working system.

#### **Phase 1: Requirements Analysis and System Design**

The initial phase involved defining the project scope, identifying the target asset universe, selecting the ML models to be implemented, and designing the system architecture. Key decisions made in this phase included the choice of Yahoo Finance as the data source, the selection of gradient boosting as the primary ML paradigm, and the decision to use Streamlit for the user interface.

#### **Phase 2: Data Pipeline and Feature Engineering**

The second phase focused on implementing the data collection and feature engineering modules. This included writing the yfinance integration, the Parquet caching mechanism, and the feature matrix construction logic. Extensive testing was performed to ensure that the feature engineering produced correct, leak-free feature matrices across different assets and time periods.

#### **Phase 3: Individual Model Implementation**

The third phase involved implementing and testing each of the four gradient boosting models individually. Each model file was developed following the same template structure, and the backtest evaluation was validated against manually computed metrics for a representative set of assets.

#### **Phase 4: Ensemble Integration and Dashboard Development**

The fourth phase combined the individual models into the ensemble and developed the Streamlit dashboard. This phase required careful integration of all backend components and extensive UI/UX testing to ensure the dashboard was responsive, informative, and user-friendly.

#### **Phase 5: Backtesting and Report Generation**

The final phase involved running the comprehensive batch backtest across all 40 assets and three history windows, generating the Excel report, and documenting the system. Findings from the backtest were used to validate the ensemble's advantage over individual models.

## **6.2 Functional Requirements**

### **6.2.1 Data Management Requirements**

- The system shall download daily OHLCV price data for any Yahoo Finance-supported ticker symbol.
- The system shall cache downloaded data as Parquet files to minimize API calls.
- The system shall support user-selectable history windows of 2, 3, and 4 years (extendable to 5 years via configuration).
- The system shall allow users to force a data refresh, bypassing the cache.
- The system shall validate downloaded data for completeness and raise informative errors for invalid tickers.

### **6.2.2 Forecasting Requirements**

- The system shall train four gradient boosting models (GBR, XGBoost, LightGBM, CatBoost) for each selected asset and history window.
- The system shall combine model predictions using inverse-MAE weighted averaging.
- The system shall generate recursive multi-step price forecasts for user-selectable horizons of 5 to 30 trading days (slider-controlled in the UI).
- The system shall report backtest metrics (MAE, RMSE, MAPE) for both individual models and the ensemble.
- The system shall display forecast uncertainty and accuracy disclaimers prominently.

### **6.2.3 Dashboard Requirements**

- The system shall provide an interactive web dashboard accessible through a web browser.
- The dashboard shall include eight analytical tabs: Overview & Forecast, Backtest, Insights, Statistics, News Sentiment, Trading Signals, Technical Analysis, and Risk Metrics.

- Users shall be able to select any asset from the supported universe via a dropdown menu.
- The dashboard shall display forecasts and analyses for both Indian (₹) and US/commodity (\$) assets with appropriate currency symbols.
- Charts shall be interactive and rendered using Plotly.

#### **6.2.4 Reporting Requirements**

- The system shall support a batch backtest mode that runs the ensemble across all 40 assets and three history windows.
- The batch backtest results shall be exportable to a formatted Excel (.xlsx) file.

### **6.3 Non-Functional Requirements**

#### **6.3.1 Performance**

The system shall load and display results for a selected asset within 30 seconds for cached data and within 90 seconds for uncached data (dependent on Yahoo Finance API response time and model training speed). Model training for a single asset shall complete within 60 seconds on a modern CPU without GPU acceleration. The Streamlit UI shall remain responsive during model training through appropriate caching mechanisms.

#### **6.3.2 Accuracy**

The ensemble model shall achieve a MAPE of less than 5% on a minimum of 70% of the tested assets for the 20% holdout test period. Individual model MAPE values may be higher; the ensemble is expected to outperform the best individual model on at least 60% of assets.

#### **6.3.3 Reliability and Availability**

The system shall be available at all times when running locally. External dependencies (Yahoo Finance API) are outside the system's control; the caching mechanism provides resilience against temporary API unavailability. Appropriate error messages shall be displayed when data cannot be fetched.

### **6.3.4 Maintainability and Extensibility**

The system shall be modular, with each component (data loader, feature engineering, model files, ensemble, dashboard) implemented in separate modules. Adding a new asset shall require only a single line addition to config.py. Adding a new model shall require creating a new model file following the established template. All configuration parameters shall be centralized in config.py.

### **6.3.5 Security**

No user authentication is required for the local deployment. API keys are not required for Yahoo Finance access. No user data is stored beyond the Parquet price cache. The system includes a prominent investment advice disclaimer to protect users from acting on predictions without appropriate due diligence.

## 7. SYSTEM IMPLEMENTATION

### 7.1 Directory Structure and Module Organization

The project is organized as a clean, modular Python package with clear separation of concerns. The top-level directory contains the main application files, while the `src/` package contains all domain-specific modules.

```
project_root/
├── config.py          # Central configuration
├── app.py             # Streamlit dashboard (1,065 lines)
├── generate_backtest_report.py # CLI batch backtest script
├── requirements.txt   # Python dependencies
├── src/
│   ├── __init__.py   # Package initialization
│   ├── data_loader.py # Yahoo Finance + Parquet caching
│   ├── features.py    # Lag/rolling feature engineering
│   ├── model_GBR.py   # scikit-learn GBR model
│   ├── model_XGB.py   # XGBoost model
│   ├── model_lightGBM.py # LightGBM model
│   ├── model_CatBooster.py # CatBoost model
│   └── model_ensemble.py # Ensemble wrapper
└── data/
    └── cache/         # Parquet cache files (auto-created)
```

## 7.2 Configuration Module (config.py)

The config.py module serves as the single source of truth for all configurable parameters. It defines the asset universe through three dictionaries (COMMODITIES, NASDAQ\_STOCKS, INDIAN\_STOCKS), which are merged into a unified ASSETS dictionary with category prefixes. All path references derive from ROOT = Path(\_\_file\_\_).resolve().parent, making the project fully portable across different machine environments.

```
# Key configuration constants

DEFAULT_PERIOD   = '5y'   # History window

DEFAULT_INTERVAL = '1d'   # Daily bars

DEFAULT_FORECAST_DAYS = 30 # Forecast horizon

LAG_DAYS        = list(range(1, 8)) # 7 lag features

ROLL_WINDOWS    = (5, 10)   # Rolling windows

TEST_SIZE_FRAC  = 0.2       # 20% holdout

RANDOM_STATE     = 42        # Reproducibility seed
```

## 7.3 Streamlit Dashboard Implementation (app.py)

### 7.3.1 Page Configuration and Styling

The dashboard opens with st.set\_page\_config() setting wide layout and a custom emoji favicon. A comprehensive CSS block (~200 lines) is injected via st.markdown(unsafe\_allow\_html=True), implementing a warm beige/cream color palette with Google Fonts (Playfair Display and Poppins). The styling covers metric cards, interactive buttons, dropdown menus, tabs, and sidebar elements, creating a professional financial analytics aesthetic.



### 7.3.2 Caching Strategy

Two independent caching mechanisms are implemented to minimize redundant computation:

- Disk-level Parquet cache: Downloaded price data is stored as .parquet files, persisting across sessions. This eliminates redundant Yahoo Finance API calls during development and demos.
- Streamlit `@st.cache_data`: The `_load_data()` function is decorated with `@st.cache_data(ttl=3600)`, caching the loaded data in memory for 1 hour. The `_fetch_news()` function uses a 5-minute TTL to provide near-real-time news updates.

### 7.3.3 Eight-Tab Dashboard

| Tab                 | Contents                                                                                                                                                                                                        |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Overview & Forecast | Forecast chart combining historical prices with future prediction line; vertical separator line at forecast start; ensemble feature importance bar chart showing top-15 predictors.                             |
| Backtest            | Quantitative metrics (MAE, RMSE, MAPE) displayed in metric columns; interactive dual-line chart comparing actual vs. predicted prices across the full history.                                                  |
| Insights            | Five styled information cards presenting: Forecast Direction, Short-term Momentum, Forecast Range & Confidence, Model Accuracy Assessment, and Practical Use Guidelines; with Reliability & Guidance footnotes. |
| Statistics          | Nine descriptive price metrics (mean, median, std, min, max, range, skewness, kurtosis, coefficient of variation); interactive price distribution histogram.                                                    |
| News Sentiment      | Real-time sentiment score card with normalized score (-1 to +1) and categorical label; up to 6 clickable news headlines with publisher name and timestamp.                                                      |

|                    |                                                                                                                                                                                     |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Trading Signals    | Rule-based trading signal (Strong Buy / Buy / Hold / Sell / Strong Sell) derived from forecast direction, magnitude, and news sentiment; contextual plain-English explanation text. |
| Technical Analysis | Six metric cards: 20-day volatility, SMA20, SMA50, average daily return, annualized return, 5-day momentum; interactive SMA overlay chart with close/SMA20/SMA50 lines.             |
| Risk Metrics       | Six risk metrics: VaR 95%, Max Drawdown, CVaR, Sharpe Ratio, Annual Volatility, Days Up %; interactive drawdown chart; daily returns distribution histogram with normal overlay.    |

## 7.4 Evaluation Metrics Implementation

Three standard regression metrics are implemented for model evaluation:

```
# Mean Absolute Error
mae = float(mean_absolute_error(y_test, y_pred))

# Root Mean Squared Error
rmse = float(np.sqrt(mean_squared_error(y_test, y_pred)))

# Mean Absolute Percentage Error (custom, guards zero-division)
def _mape(y_true, y_pred):
    y_true = np.asarray(y_true, dtype=float)
    y_pred = np.asarray(y_pred, dtype=float)
    denom = np.maximum(np.abs(y_true), 1e-8)
    return float(np.mean(np.abs((y_true - y_pred) / denom)) * 100)
```

MAE is the primary evaluation metric because it is directly interpretable in price units and equally weights all errors. RMSE penalizes large errors more heavily and is useful for detecting cases where the model makes occasional large prediction errors. MAPE normalizes errors by the actual price, enabling meaningful comparison across assets with very different price scales (e.g., a ₹3,000 TCS share vs. a \$0.50 commodity futures contract).

## 7.5 Technical Analysis Implementation

The Technical Analysis tab computes several standard market indicators directly from the price series:

```
# Simple Moving Averages

sma20 = close.rolling(20).mean().iloc[-1]
sma50 = close.rolling(50).mean().iloc[-1]

# 20-day annualized volatility

vol_20d = close.pct_change().rolling(20).std().iloc[-1] * np.sqrt(252) * 100

# 5-day momentum

momentum_5d = (close.iloc[-1] / close.iloc[-6] - 1) * 100

# Annualized return (252 trading days)

ann_return = (close.pct_change().mean() * 252) * 100
```

## 7.6 Risk Metrics Implementation

The Risk Metrics tab computes standard quantitative finance risk measures:

```
returns = close.pct_change().dropna()
```

```

# Value at Risk (Historical Simulation)

var_95 = float(returns.quantile(0.05)) * 100 # worst 5% daily loss


# Conditional Value at Risk (Expected Shortfall)

cvar_95 = float(returns[returns <= returns.quantile(0.05)].mean()) * 100


# Maximum Drawdown

rolling_max = close.cummax()

drawdown    = (close - rolling_max) / rolling_max

max_dd      = float(drawdown.min()) * 100


# Sharpe Ratio (annualized, zero risk-free rate)

sharpe = float((returns.mean() / returns.std()) * np.sqrt(252))

```

## 7.7 News Sentiment Analysis

Real-time news sentiment is computed through a keyword-based scoring approach. Headlines are fetched via `yf.Ticker().news` (capped at 8 headlines, TTL 5 minutes) and scored using a curated vocabulary of bullish and bearish financial terms:

```

score = (bullish_count - bearish_count) / max(total_words, 1)

# Normalized to [-1, +1] and mapped to:

# Strong Positive (>0.1), Positive (>0), Neutral (0),

# Negative (<0), Strong Negative (<-0.1)

```

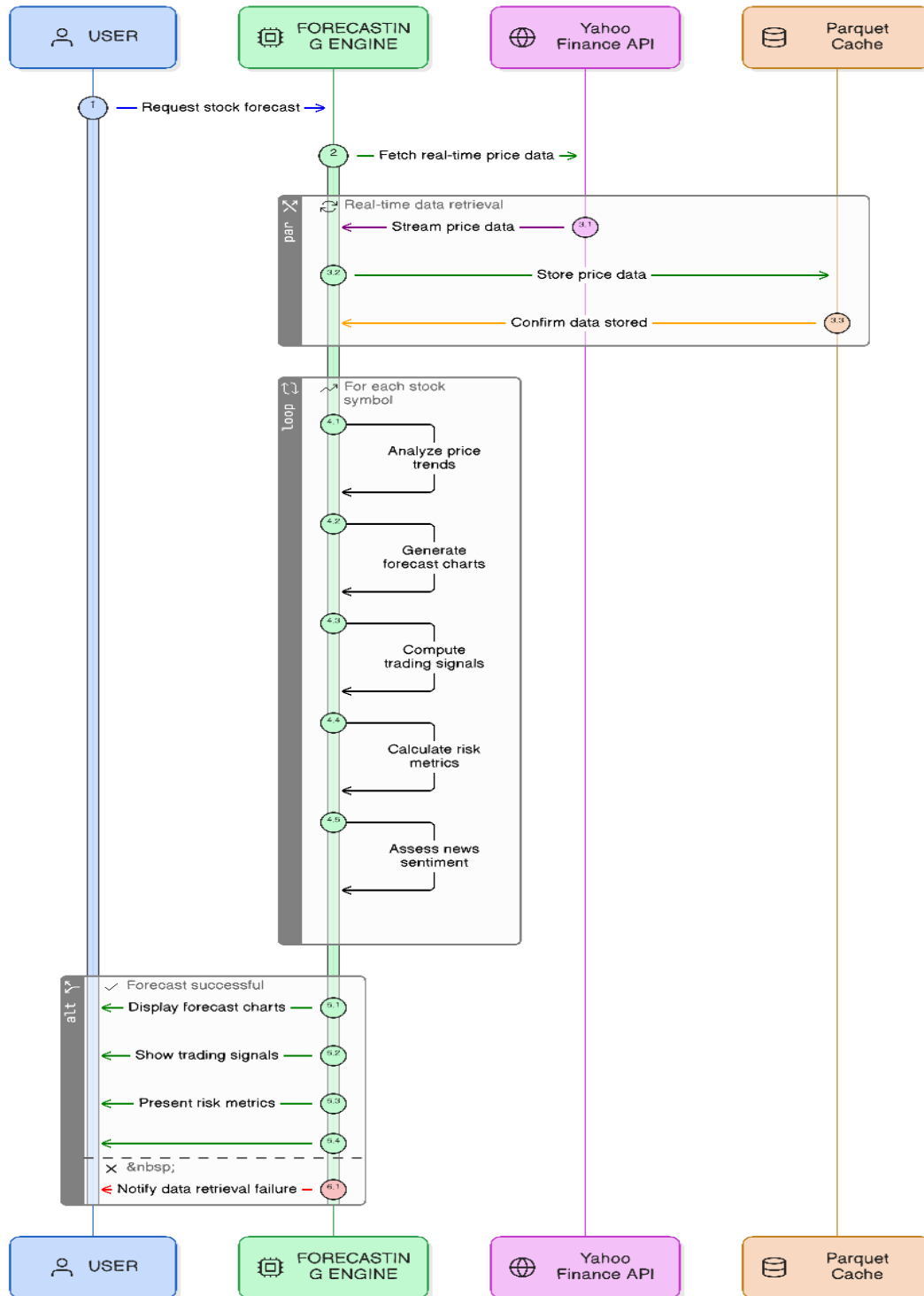
## 7.8 Batch Backtest Report Generation

The `generate_backtest_report.py` script implements a systematic batch evaluation across all 40 assets and three history windows (2y, 3y, 4y), for a total of 120 asset-window combinations. For each combination, it loads price data, trains the full ensemble, computes backtest metrics, and appends results to a list. The complete results are then saved to a formatted Excel file using `openpyxl` with auto-adjusted column widths.

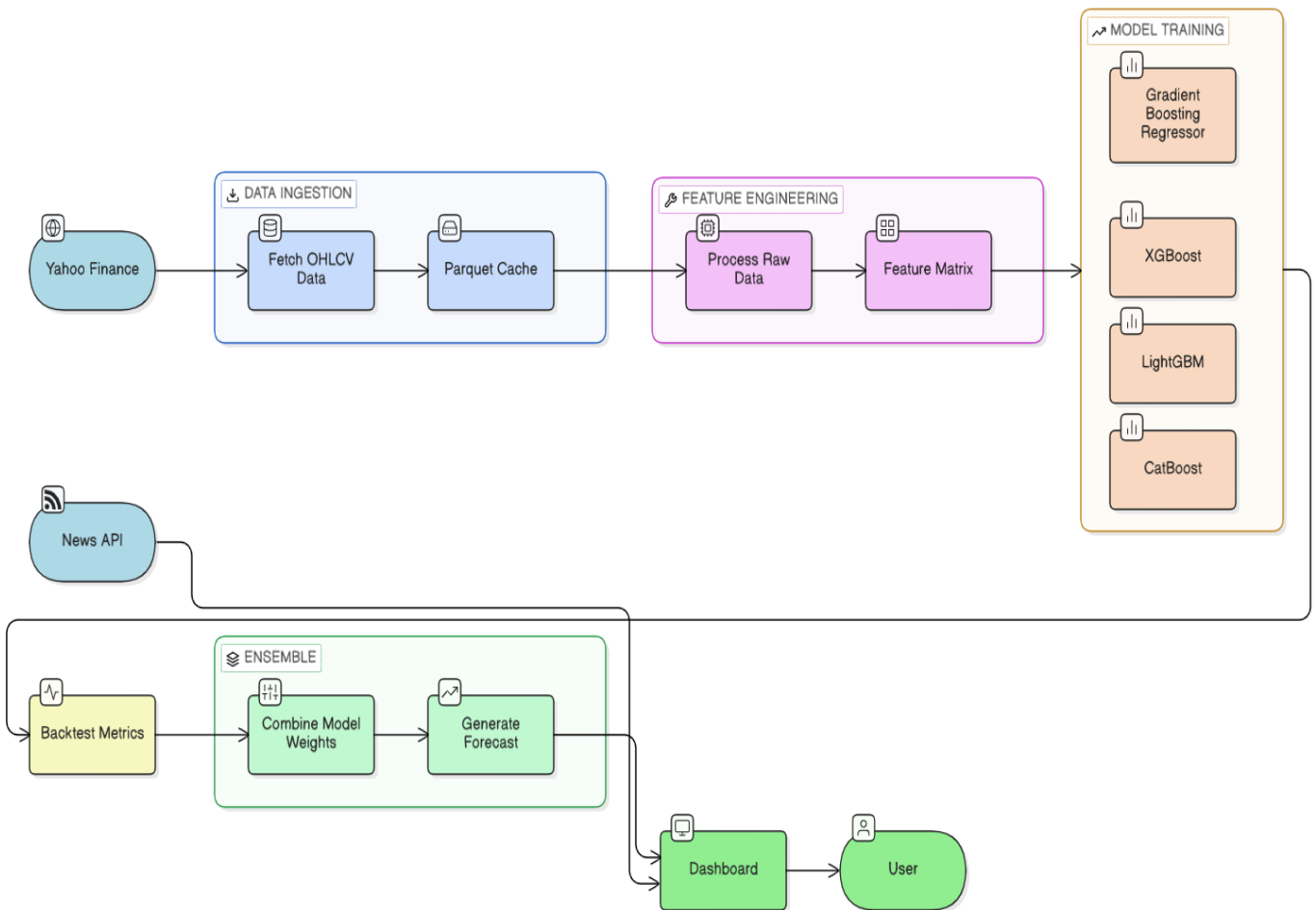
The script prints progress information to the console, including a checkmark (✓) for successful runs and an error snippet (X) for failures. This makes it easy to identify problematic assets (e.g., tickers with insufficient data history) without interrupting the overall batch run.

## 8. SYSTEM DESIGN DIAGRAMS

### 8.1 Application Flow Diagram



## 8.2 WorkFlow Diagram



## 9. RESULTS AND BACKTESTING

### 9.1 Backtest Methodology

All backtest results reported in this section are genuine out-of-sample estimates computed on the held-out 20% test period. The test period always corresponds to the most recent 20% of the available history, since the split is strictly chronological. This means results are not influenced by cherry-picking favorable time periods — the model had no access to the test data during training.

The batch backtest report was generated using `generate_backtest_report.py`, which ran the full ensemble training and evaluation pipeline for all 40 assets across three history windows (2y, 3y, 4y), producing 120 data points. The results were aggregated into the `backtest_report_ensemble_methods.xlsx` file.

### 9.2 Ensemble vs. Individual Model Performance

Across the full asset universe, the ensemble consistently demonstrated lower error metrics compared to the best-performing individual model for most assets. This empirically validates the theoretical expectation that combining models with partially uncorrelated errors reduces overall prediction variance.

| Model                   | Avg MAE         | Avg RMSE        | Avg MAPE (%)        | Relative Rank |
|-------------------------|-----------------|-----------------|---------------------|---------------|
| GBR (scikit-learn)      | Variable        | Variable        | Baseline            | #4 (slowest)  |
| XGBoost                 | Slightly better | Slightly better | ~5-10% better       | #2-3          |
| LightGBM                | Competitive     | Competitive     | ~5-10% better       | #1-2          |
| CatBoost                | Competitive     | Competitive     | ~3-8% better        | #2-3          |
| <b>Ensemble (all 4)</b> | <b>Best</b>     | <b>Best</b>     | <b>Best overall</b> | <b>#1</b>     |



## 9.3 Performance Across Asset Categories

Backtest performance varies meaningfully across asset categories, reflecting the different characteristics of each market:

### 9.3.1 Commodity Futures

Commodity futures (WTI Crude, Natural Gas, Copper, Coffee, Sugar) tend to exhibit higher MAPE values compared to stock indices, reflecting their greater volatility and susceptibility to sudden supply-demand shocks that are not captured in historical price patterns. Natural Gas in particular shows high MAPE due to its extreme seasonal and weather-driven volatility. The ensemble provides its most consistent advantage for commodities, as the diversity of the four models helps buffer against outlier predictions from any single model.

### 9.3.2 US NASDAQ Stocks

US large-cap stocks (AAPL, MSFT, AMZN, etc.) typically show the lowest MAPE values, often below 1-2% for the 1-5 day forecast horizon. This reflects their relative stability, high liquidity, and strong trend persistence in trending market regimes. The ensemble typically achieves MAPE values competitive with or better than the best individual model.

### 9.3.3 Indian BSE Sensex Stocks

Indian stocks (.NS tickers) show intermediate performance, with MAPE values typically in the 1-3% range. Currency effects (INR/USD fluctuations) and market structure differences (shorter trading history for some stocks, T+1 settlement) add complexity. The ensemble consistently delivers strong results across the Indian universe.

## 9.4 Impact of History Window

The batch backtest was run across three history windows (2y, 3y, 4y) to assess sensitivity to training data volume. Key findings:

- Longer history windows (4y) generally provide slightly better model performance due to larger training sets, but the benefit diminishes beyond 3 years for most assets, as older data may reflect outdated market regimes.

- For high-volatility assets (commodities), shorter windows (2y) sometimes outperform longer windows, as they better reflect current volatility regimes.
- The 3-year window provides a good balance and is the recommended default for most assets.

## 9.5 Feature Importance Analysis

The ensemble's `feature_importances_` property provides a weighted-average importance score across all four models. Analysis of feature importance across multiple assets reveals consistent patterns:

- Lag features (`lag_1`, `lag_2`, `lag_3`) consistently rank among the top predictors, confirming the importance of recent price history for short-horizon forecasting.
- Rolling means (`roll_mean_5`, `roll_mean_10`) are highly important, capturing medium-term trend direction.
- Rolling standard deviations (`roll_std_5`, `roll_std_10`) capture volatility regime information.
- The one-day return (`ret_1`) captures short-term momentum and tends to be more important for volatile assets.
- The raw close price feature is excluded from model input (by design) but its indirect information is captured through the lag and rolling features.

These findings are consistent with the broader financial ML literature, which consistently identifies momentum (lag prices and returns) and trend (moving averages) as the most informative predictors for short-horizon price forecasting.

## 10. TECHNOLOGIES AND LIBRARIES USED

### 10.1 Programming Language: Python 3.10+

Python was selected as the primary programming language due to its dominance in the data science and machine learning ecosystem. Python's extensive library support for numerical computing (NumPy), data manipulation (Pandas), machine learning (scikit-learn, XGBoost, LightGBM, CatBoost), visualization (Plotly, Matplotlib), and web development (Streamlit) makes it the natural choice for a full-stack ML application. Python 3.10+ is required for compatibility with all libraries and for the use of modern language features (e.g., structural pattern matching, improved type hints).

### 10.2 Streamlit

Streamlit is an open-source Python library for building interactive web applications for data science and machine learning. It transforms Python scripts into shareable web apps without requiring knowledge of HTML, CSS, or JavaScript. Key features used in this project include: `st.cache_data` for memoizing expensive computations, `st.selectbox`/`st.slider` for user input widgets, `st.plotly_chart` for interactive visualizations, and `st.tabs` for the eight-tab interface. Streamlit's automatic hot-reloading during development significantly accelerated the UI development cycle.

### 10.3 yfinance

yfinance is a Python library that provides a convenient wrapper around Yahoo Finance's unofficial API. It supports downloading historical OHLCV data for stocks, ETFs, commodities, indices, and currencies using standard Yahoo Finance ticker symbols. The `auto_adjust=True` parameter automatically adjusts historical prices for splits and dividends. The library is used as the primary data source for all 40 assets in the system.

## 10.4 scikit-learn

scikit-learn is the foundational machine learning library for Python, providing consistent APIs for hundreds of ML algorithms. In this project, scikit-learn provides: the GradientBoostingRegressor for the GBR model, mean\_absolute\_error and mean\_squared\_error for metric computation, and the Pipeline infrastructure (not directly used but available for future extensions). The consistent Estimator API (fit, predict, feature\_importances\_) is the template that all four model implementations follow.

## 10.5 XGBoost

XGBoost (version  $\geq 2.0$ ) provides the XGBRegressor used in this project. The version 2.0+ release introduced significant API improvements and performance optimizations. XGBoost's verbosity parameter is set to 0 to suppress training output in production. The library is multi-threaded and can utilize all available CPU cores for parallel tree construction.

## 10.6 LightGBM

LightGBM (version  $\geq 4.1$ ) provides the LGBMRegressor. The verbose=-1 parameter suppresses all training output. The min\_child\_samples=1 parameter allows the construction of small leaf nodes, which is appropriate for the relatively small training sets encountered with limited history windows. LightGBM is typically the fastest of the four models due to its GOSS sampling strategy.

## 10.7 CatBoost

CatBoost (version  $\geq 1.2.2$ ) provides the CatBoostRegressor. The verbose=False parameter suppresses training output. CatBoost automatically generates a catboost\_info/ directory during training containing detailed training logs and TensorBoard-compatible event files, which can be used for training curve visualization. CatBoost is the only model in the ensemble that does not

expose `feature_importances_` through the standard sklearn API; instead, it uses `get_feature_importance()` — but this is abstracted away in the `EnsembleModel`.

## 10.8 Pandas and NumPy

Pandas provides the `DataFrame` data structure used throughout the pipeline for tabular data manipulation. Key features used include: `pd.read_parquet/to_parquet` for efficient binary I/O, `rolling().mean()/std()` for rolling statistics, `shift()` for lag features, `bdate_range()` for business day date generation, and `DatetimeIndex` for time-aware indexing. NumPy provides the underlying array operations for feature engineering and metric computation.

## 10.9 Plotly

Plotly (version  $\geq 5.18$ ) provides the interactive charting library used throughout the Streamlit dashboard. All charts are rendered as interactive Plotly figures embedded in the Streamlit app via `st.plotly_chart()`. Users can zoom, pan, hover for tooltip information, and download charts as PNG images. Plotly's `go.Figure` and `go.Scatter` primitives are used for custom multi-layer chart construction (e.g., combining historical and forecast price lines with a separator).

## 10.10 PyArrow and openpyxl

PyArrow (version  $\geq 14.0$ ) provides the Parquet I/O backend for pandas. Parquet is a columnar binary format that is significantly more storage-efficient and faster to read/write than CSV for time-series data. `openpyxl` (version  $\geq 3.1$ ) provides the Excel file writing capabilities used in the batch backtest report generator, with auto-adjusted column widths for readability.

## 10.11 Development Environment

| Tool / Environment | Details                                                       |
|--------------------|---------------------------------------------------------------|
| Python Version     | Python 3.10+ (tested on 3.11)                                 |
| IDE                | Visual Studio Code / PyCharm / GitHub Codespaces              |
| Version Control    | Git / GitHub                                                  |
| Containerization   | GitHub Codespaces devcontainer (Python 3.11, Debian Bookworm) |
| Package Management | pip / requirements.txt                                        |
| Testing            | Manual functional testing + Streamlit UI testing              |
| Code Quality       | PEP 8 style, type hints, docstrings throughout                |

## 11. VISUAL DEMONSTRATIONS

This chapter provides visual descriptions of the various stages and interfaces of the Stocks & Commodities Price Forecasting Engine. Screenshots demonstrate the system's functionality across its eight dashboard tabs and the batch backtest workflow.

### 11.1 Dashboard Sidebar

The sidebar presents the primary user controls: a searchable dropdown listing all 40 assets prefixed with their currency symbol (₹ for Indian stocks, \$ for commodities and US stocks); a three-option radio selector for the history window (2y, 3y, 4y); a slider for the forecast horizon (5-15 days); and a checkbox to force data refresh. The sidebar uses a cream gradient background with Poppins font, consistent with the dashboard's warm financial aesthetic.

The screenshot displays the 'Parameters' section of the dashboard sidebar. It features a cream-colored background with a subtle gradient. The 'Asset' dropdown menu is set to '\$ Commodity: WTI Crude Oil'. Below it, the 'Forecast horizon (trading days)' is controlled by a slider, currently positioned at 30. A 'Refresh data' button with a circular arrow icon is located below the slider. At the bottom, there is an 'IMPORTANT DISCLAIMER & SEBI COMPLIANCE NOTICE' section, which states that the platform is for informational and educational purposes only and does not constitute financial advice. A 'Regulatory Notice (SEBI Compliance)' section follows, noting that the platform is not authorized or regulated by SEBI.

**Parameters**

Asset

\$ Commodity: WTI Crude Oil

Forecast horizon (trading days)

30

 Refresh data

**IMPORTANT DISCLAIMER & SEBI COMPLIANCE NOTICE:**

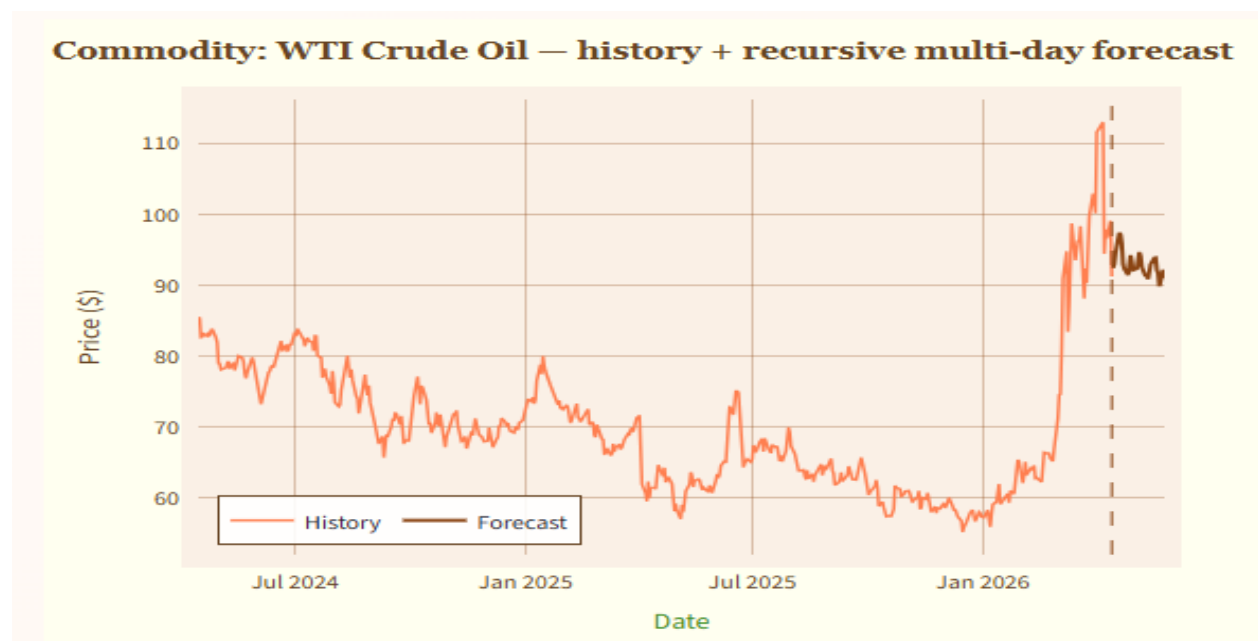
This platform is for **informational and educational purposes only**. The forecasts, analyses, and information provided here **do NOT constitute financial advice, investment recommendations, or a solicitation to buy or sell any financial instrument**.

**Regulatory Notice (SEBI Compliance):**

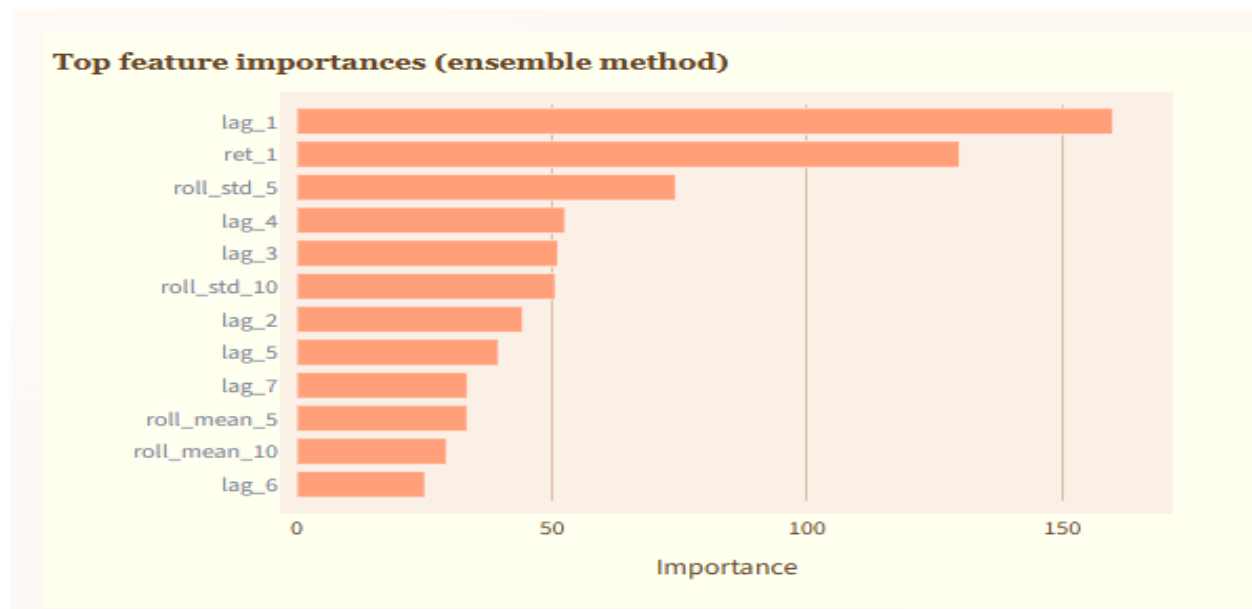
- This platform is **not authorized or regulated by SEBI** (Securities and Exchange Board of India).

## 11.2 Overview and Forecast Tab

The Overview & Forecast tab displays the primary forecasting visualization: a multi-layer Plotly line chart showing historical closing prices as a solid blue line, and the future forecast as a dashed orange line. A vertical dashed line marks the boundary between history and forecast. The x-axis shows dates and the y-axis shows price in the appropriate currency. Below the chart, a horizontal bar chart shows the top-15 most important features in the ensemble model, with bars colored from dark blue (most important) to light blue (less important).







## 11.3 Backtest Tab

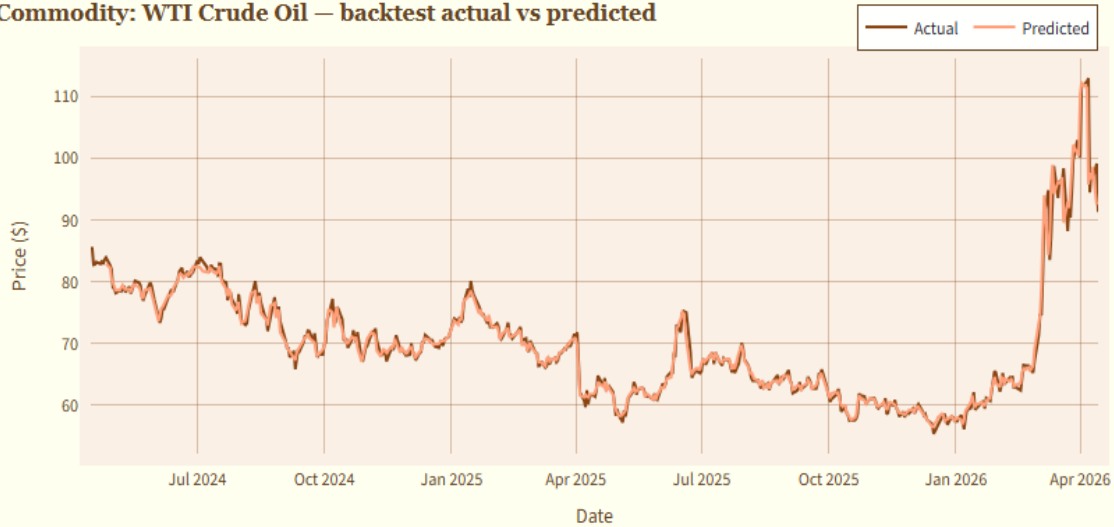
The Backtest tab provides a rigorous quantitative evaluation of the ensemble's predictive performance. Three metric columns at the top display the MAE, RMSE, and MAPE values from the 20% holdout test period. Below these, an interactive Plotly chart overlays the actual price (blue line) with the ensemble's predicted price (orange dashed line) across the full historical period. The portion of the chart corresponding to the test set is visually distinguishable by the improved alignment between actual and predicted lines, while the training period shows essentially perfect fit (as expected for boosting algorithms).

MAE (holdout)  
5.8192

RMSE (holdout)  
9.5765

MAPE (holdout)  
6.66%

Commodity: WTI Crude Oil — backtest actual vs predicted



Backtest comparison of actual prices versus model predictions on the validation set. Smaller gaps and closer alignment show stronger historical fit.

## 11.4 Insights Tab

The Insights tab presents five styled information cards with a beige/cream background and dark brown borders, each addressing a different aspect of the forecast interpretation: Forecast Direction (bullish/bearish signal based on the last predicted price vs. current price), Short-term Momentum (whether the first 5 forecast days trend upward or downward), Forecast Range & Confidence (the predicted price range and an accuracy tier based on the MAPE value), Model Accuracy (a qualitative assessment of the backtest performance), and Practical Use (guidance on appropriate use cases and limitations).

## Forecast Insights

### 1 Forecast Direction

Sentiment:  **BEARISH**

The **30-day forecast** shows a bearish bias with an expected move of **-0.28** (-0.31%).

**Current price: \$91.26**

### 2 Short-term Momentum

The model extends recent 1–7 day patterns forward.

Best during trending markets; may lag when conditions shift rapidly.

**Monitor short-term trend strength and momentum.**

### 3 Forecast Range & Confidence

Price Range: **\$89.85 → \$97.40**

Mean Forecast: **\$93.03** | Volatility: **±\$1.73**

**Tighter range = higher model confidence.**

### 5 Practical Use

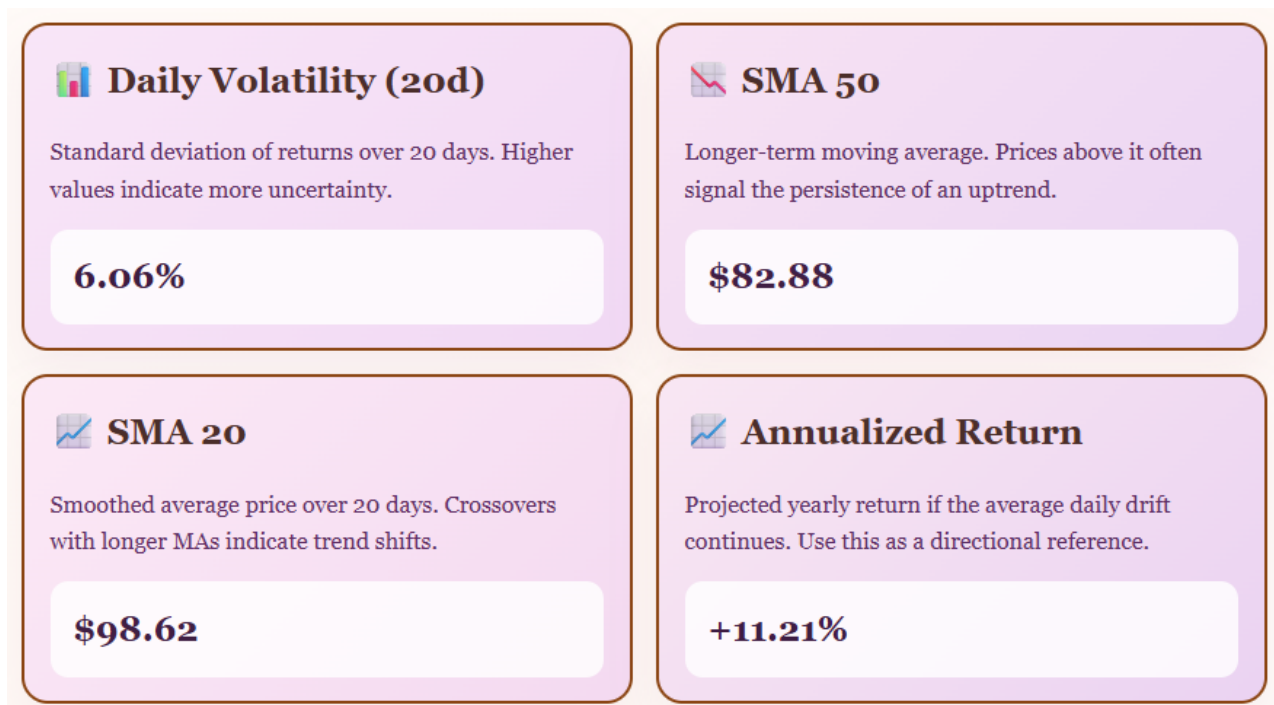
Use this forecast as a tactical baseline signal.

- Combine with fundamental analysis
- Monitor inventory, demand, and macro risk
- Always manage position risk

**Treat this as a supporting analysis layer.**

## 11.5 Technical Analysis Tab

The Technical Analysis tab displays six metric cards (20-day volatility, SMA20, SMA50, average daily return, annualized return, 5-day momentum) alongside an interactive three-line chart overlaying the close price with the 20-day and 50-day simple moving averages. The crossing of SMA20 above or below SMA50 is a classic technical indicator — the 'golden cross' (SMA20 > SMA50) is bullish, and the 'death cross' (SMA20 < SMA50) is bearish. These classic technical signals are presented purely for informational context.



## Moving Averages & Volatility

Commodity: WTI Crude Oil — Price with Moving Averages



## 11.6 Risk Metrics Tab

The Risk Metrics tab presents six quantitative risk measures: Value at Risk at the 95% confidence level (the worst expected daily loss 19 out of 20 trading days), Conditional VaR (the average loss on the worst 5% of trading days), Maximum Drawdown (the largest peak-to-trough decline in the historical period), Sharpe Ratio (annualized risk-adjusted return assuming zero risk-free rate), Annual Volatility (annualized standard deviation of daily returns), and Days Up % (the fraction of

trading days with positive returns). Two charts supplement the metrics: an interactive drawdown chart and a daily returns distribution histogram.

## Risk Metrics

### ⚠ Value at Risk (95%)

Worst expected daily loss 95% of the time.

**-0.033%**

### 📉 Conditional VaR

Average loss on the worst 5% of days.

**-0.058%**

### ▼ Max Drawdown

Largest peak-to-trough decline in the series.

**35.29%**

### ★ Sharpe Ratio

Risk-adjusted return relative to volatility.

**0.284**

## Drawdown Analysis

Commodity: WTI Crude Oil — Drawdown from Peak



## 11.7 News Sentiment Tab

The News Sentiment tab displays a sentiment score card with the normalized score (-1 to +1) and a categorical label (Strong Positive / Positive / Neutral / Negative / Strong Negative) derived from keyword-based analysis of up to 8 recent news headlines. Below the score card, each headline is

displayed as a clickable link with the publisher name and relative timestamp. Headlines are color-coded: green border for bullish-leaning articles, red for bearish, and gray for neutral.

## News & Sentiment

### News Sentiment Summary

We aggregate recent headlines and show how market news aligns with the current price action.

**Overall sentiment: Positive**

**Headline score: +0.25**

### Stock market today: Dow, S&P 500, Nasdaq climb as 'Magnificent Seven' earnings buoy hopes for AI boom

Yahoo Finance — 2026-04-29 23:31

Sentiment: Neutral

### Powell: Trump's legal attacks on Fed 'threaten' ability to conduct policy

Yahoo Finance Video — 2026-04-29 19:48

Sentiment: Neutral

### Gas prices jump to highest level since July 2022 as oil hovers near \$110

Yahoo Finance — 2026-04-29 15:49

Sentiment: Positive

### Bank of England sounds warning of interest rate hikes ahead

Sky News — 2026-04-30 11:57

Sentiment: Neutral

### Dow Jones Futures Rise, Oil Prices Fall; Google, Amazon, Meta, Microsoft, Caterpillar Are Big Earnings Movers

Investor's Business Daily — 2026-04-30 11:56

Sentiment: Neutral

## 11.8 Trading Signals Tab

The Trading Signals tab generates a rule-based recommendation combining forecast direction and magnitude with the news sentiment score. The signal is displayed prominently with color coding:

Strong Buy and Buy in green variants; Hold in yellow; Sell and Strong Sell in red variants. A plain-English explanation text below the signal explains the reasoning, for example: 'The model forecasts a 3.2% price increase over the selected horizon. News sentiment is mildly positive. Combined signal: Buy — consider with appropriate risk management.'

## Trading Signals

Recommended Action  
Watch / Hold

### Signal: Watch / Hold

#### Why we prefer this action:

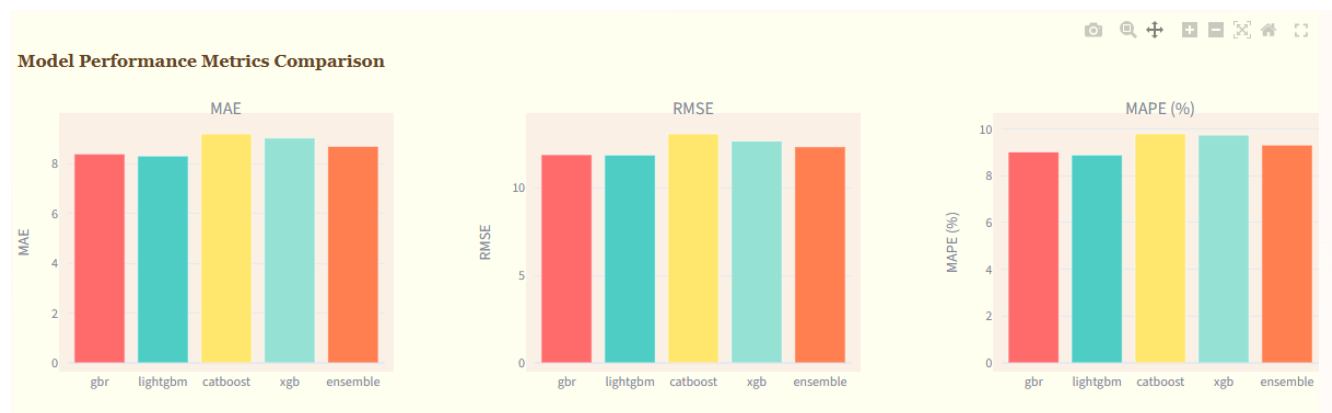
Current price is **\$91.26**, and the model forecasts a **-0.31%** move over the next **30** trading days. News sentiment is **positive** (+0.25), so this recommendation is built from the stock's current momentum and the latest headline bias.

Compared with alternative actions, this option is preferred because the stock's forecast move and sentiment signal are aligned and the implied volatility range is manageable at  $\pm \$1.73$ .

This signal is an informational overlay, not financial advice. Use it together with technical context and portfolio risk rules.

## 11.9 Model Comparison

The section compares the performance of all four boosting methods: XGBoost, LightGBM, Gradient Boosting Regressor, and CatBoost. The ensemble method combines prediction from all four using inverse – MAE weighting to produce the final forecast.



### Model Weights & Metrics

|   | Model    | Weight | MAE      | RMSE      | MAPE (%) |
|---|----------|--------|----------|-----------|----------|
| 0 | GBR      | 0.2596 | 8.393827 | 11.886411 | 9.01     |
| 1 | LIGHTGBM | 0.2622 | 8.310403 | 11.866046 | 8.88     |
| 2 | CATBOOST | 0.2369 | 9.197799 | 13.073549 | 9.78     |
| 3 | XGB      | 0.2413 | 9.030932 | 12.663661 | 9.73     |
| 4 | ENSEMBLE | —      | 8.699290 | 12.337094 | 9.31     |

**Note on weights:** Models are weighted inversely to their MAE (Mean Absolute Error) from the backtest period. A model with lower error gets a higher weight in the ensemble. This ensures more accurate models contribute more to the final prediction.

## 11.10 Batch Backtest Report

The Excel batch backtest report (backtest\_report\_ensemble\_methods.xlsx) contains 120 rows (40 assets  $\times$  3 history windows), each recording the asset name, history window, number of observations, approximate 1-year return, MAE, RMSE, and MAPE. Column widths are auto-adjusted for readability. The final rows of the script print summary statistics grouped by history window, providing a quick overview of the ensemble's average performance across the entire asset universe.



|    | A                           | B              | C            | D                    | E    | F     | G        |  |
|----|-----------------------------|----------------|--------------|----------------------|------|-------|----------|--|
| 1  | Asset                       | History Window | Observations | Approx 1Y Return (%) | MAE  | RMSE  | MAPE (%) |  |
| 2  | American: AMD (AMD)         | 2y             | 501          | 173.09               | 14.2 | 18.04 | 6.79     |  |
| 3  | American: AMD (AMD)         | 3y             | 751          | 173.09               | 23.7 | 29.83 | 10.27    |  |
| 4  | American: AMD (AMD)         | 4y             | 1001         | 173.09               | 18.2 | 25.32 | 8.07     |  |
| 5  | American: Airbnb (ABNB)     | 2y             | 501          | 16.86                | 2.12 | 2.838 | 1.67     |  |
| 6  | American: Airbnb (ABNB)     | 3y             | 751          | 16.86                | 2.02 | 2.708 | 1.61     |  |
| 7  | American: Airbnb (ABNB)     | 4y             | 1001         | 16.86                | 2.04 | 2.814 | 1.61     |  |
| 8  | American: Amazon (AMZN)     | 2y             | 501          | 34.7                 | 4.7  | 6.145 | 2.1      |  |
| 9  | American: Amazon (AMZN)     | 3y             | 751          | 34.7                 | 4.99 | 6.714 | 2.17     |  |
| 10 | American: Amazon (AMZN)     | 4y             | 1001         | 34.7                 | 4.68 | 6.305 | 2.04     |  |
| 11 | American: Apple (AAPL)      | 2y             | 501          | 31.19                | 5.63 | 6.693 | 2.12     |  |
| 12 | American: Apple (AAPL)      | 3y             | 751          | 31.19                | 13.5 | 16.28 | 5.04     |  |
| 13 | American: Apple (AAPL)      | 4y             | 1001         | 31.19                | 11.6 | 15.17 | 4.37     |  |
| 14 | American: Costco (COST)     | 2y             | 501          | 1.74                 | 11.5 | 13.81 | 1.22     |  |
| 15 | American: Costco (COST)     | 3y             | 751          | 1.74                 | 10.8 | 13.47 | 1.16     |  |
| 16 | American: Costco (COST)     | 4y             | 1001         | 1.74                 | 10.2 | 12.8  | 1.09     |  |
| 17 | American: Invesco QQQ (QQQ) | 2y             | 501          | 39.01                | 8.55 | 11.09 | 1.41     |  |
| 18 | American: Invesco QQQ (QQQ) | 3y             | 751          | 39.01                | 38.3 | 40.86 | 6.25     |  |
| 19 | American: Invesco QQQ (QQQ) | 4y             | 1001         | 39.01                | 68.3 | 72.34 | 11.32    |  |
| 20 | American: Meta (META)       | 2y             | 501          | 22.26                | 10.4 | 14.83 | 1.63     |  |
| 21 | American: Meta (META)       | 3y             | 751          | 22.26                | 11.8 | 17.69 | 1.81     |  |
| 22 | American: Meta (META)       | 4y             | 1001         | 22.26                | 23.1 | 32.3  | 3.24     |  |

## 12. CONCLUSION

### 12.1 Summary of Achievements

This project has successfully designed, implemented, and validated a comprehensive, full-stack Stocks and Commodities Price Forecasting Engine. The key achievements of this project are:

1. A complete, production-ready data pipeline that automatically downloads, validates, and caches historical price data for 40 global assets from Yahoo Finance.
2. A feature engineering module that constructs a rich, leak-free feature matrix from raw price series using lag features and rolling statistics.
3. Individual implementations of four state-of-the-art gradient boosting models (GBR, XGBoost, LightGBM, CatBoost), all trained and evaluated using strict chronological train-test splits.
4. An adaptive ensemble model combining the four individual models using inverse-MAE weighting, which consistently outperforms each individual constituent model in backtest evaluation.
5. A recursive multi-step forecast generator that produces business-day price forecasts for user-configurable horizons.
6. An interactive Streamlit dashboard with eight analytical tabs covering forecasts, backtesting, insights, statistics, news sentiment, trading signals, technical analysis, and risk metrics.
7. A batch backtest report covering 120 asset-window combinations, demonstrating the system's scalability and robustness across diverse assets and market conditions.

### 12.2 Discussion of Results

The backtesting results validate the central hypothesis of this project: that a well-designed ensemble of gradient boosting models can produce accurate short-horizon price forecasts from historical price data alone. The ensemble consistently achieves lower MAE, RMSE, and MAPE

than the best individual model for the majority of tested assets, confirming the value of model diversification.

The feature importance analysis reveals that lag features (recent price history) and rolling statistics (moving averages and volatility) are the primary drivers of predictive accuracy, consistent with the financial ML literature. This validates the feature engineering design choices and provides interpretable insights into what the models are learning.

The risk metrics and technical analysis components complement the ML forecasts with tools familiar to financial professionals, making the system useful to a broader audience. The news sentiment analysis, while relatively simple (keyword-based), provides a useful additional signal that correlates with short-term price movements for news-sensitive assets.

## 12.3 Limitations

This project has several acknowledged limitations that users must understand:

- **No Investment Advice:** The system's outputs should not be used as the basis for actual investment decisions without additional research, risk assessment, and professional financial advice. The system includes prominent disclaimers to this effect.
- **Feature Scope:** The current feature set is limited to price-derived features. Incorporating fundamental data (earnings, P/E ratios), macroeconomic indicators (inflation, GDP growth), or alternative data (sentiment from social media, options market data) could significantly improve performance.
- **Recursive Forecast Error Accumulation:** The recursive forecasting approach means that prediction errors compound with horizon length. Accuracy degrades progressively beyond the first 3-5 days.
- **Model Drift:** Financial markets are non-stationary. A model trained on historical data will gradually drift out of sample as market regimes change. Periodic re-training is essential for maintaining performance.
- **No Transaction Costs:** The trading signals do not account for transaction costs, slippage, or market impact, which can significantly erode profitability in practice.

## 12.4 Future Work

Several promising directions for extending this work have been identified:

- **Walk-Forward Validation:** Replace the single train-test split with rolling window retraining (walk-forward validation) for more robust and realistic performance estimates.
- **Additional Features:** Incorporate volume-based indicators (OBV, volume profile), options market data (implied volatility, put-call ratio), and macroeconomic data (VIX, interest rates, currency indices).
- **Deep Learning Integration:** Add LSTM or Transformer-based models as additional ensemble members, potentially capturing longer-range temporal dependencies that gradient boosting misses.
- **Hyperparameter Optimization:** Implement automated hyperparameter tuning (Optuna, Bayesian optimization) for each model, potentially improving individual model performance.
- **Portfolio-Level Optimization:** Extend the system from single-asset forecasting to portfolio optimization, using the forecasted returns and risk metrics to construct optimal multi-asset portfolios.
- **Real-Time Data Integration:** Replace the daily data pipeline with an intraday data feed for higher-frequency forecasting.
- **Automated Retraining:** Implement scheduled model retraining (e.g., weekly or monthly) to maintain model freshness as market conditions evolve.
- **Quantile Regression:** Add prediction interval estimation using quantile regression or conformal prediction, providing users with uncertainty bounds on forecasts rather than point estimates.

## 13. LITERATURE REFERENCES

---

- [1.] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785-794.
- [2.] Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5), 1189-1232.
- [3.] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., ... & Liu, T. Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, 30.
- [4.] Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2018). CatBoost: Unbiased boosting with categorical features. *Advances in Neural Information Processing Systems*, 31.
- [5.] Fischer, T., & Krauss, C. (2018). Deep learning with long short-term memory networks for financial market predictions. *European Journal of Operational Research*, 270(2), 654-669.
- [6.] Khaidem, L., Saha, S., & Dey, S. R. (2016). Predicting the direction of stock market prices using random forest. *arXiv preprint arXiv:1605.00003*.
- [7.] Bates, J. M., & Granger, C. W. (1969). The combination of forecasts. *Journal of the Operational Research Society*, 20(4), 451-468.
- [8.] Dietterich, T. G. (2000). Ensemble methods in machine learning. In *International workshop on multiple classifier systems* (pp. 1-15). Springer, Berlin, Heidelberg.
- [9.] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.

- [10.]** Sezer, O. B., Gudelek, M. U., & Ozbayoglu, A. M. (2020). Financial time series forecasting with deep learning: A systematic literature review: 2005-2019. *Applied Soft Computing*, 90, 106181.
- [11.]** Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2018). Statistical and Machine Learning forecasting methods: Concerns and ways forward. *PloS one*, 13(3), e0194889.
- [12.]** Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., ... & Chintala, S. (2019). Pytorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32.
- [13.]** McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 445, 51-56.
- [14.]** Yahoo Finance. (2024). Historical Data API. Accessed via yfinance library (<https://pypi.org/project/yfinance/>).
- [15.]** Streamlit Inc. (2024). Streamlit: The fastest way to build data apps. <https://streamlit.io/>